


Politechnika Rzeszowska

im. Ignacego Łukasiewicza

WYDZIAŁ MATEMATYKI I FIZYKI STOSOWANEJ

inż. Klaudia Zawiślak

# Budowa agenta motywowanego na bazie Dużego Modelu Językowego (LLM)

Promotor: dr inż. Marcin Kowalik

Praca magisterska
Kierunek studiów: Inżynieria i Analiza Danych

Rzeszów, 2025

# Spis treści

**Wstęp** **5**
**1. Agenci i uczenie ze wzmocnieniem** **7**
1.1 Pojęcie agenta w informatyce 7
1.2 Uczenie ze wzmocnieniem 7
1.2.1 Główne komponenty uczenia ze wzmocnieniem 8
1.3 Środowisko i cel działania agenta 9
1.4 Q-learning – klasyczny algorytm RL 10
1.5 Wady klasycznych agentów RL 13
1.6 Koncepcja agenta motywowanego 14
**2. Uczenie motywowane i agenci motywowani** **15**
2.1 Definicja i charakterystyka agenta motywowanego 15
2.2 Model i architektura agenta motywowanego 19
2.3 Podstawy teoretyczne uczenia motywowanego 25
2.4 Różnice między uczeniem motywowanym a klasycznym uczeniem ze wzmocnieniem 27
**3. Duże modele językowe (LLM) w kontekście sztucznej inteligencji** **30**
3.1 Geneza i rozwój dużych modeli językowych 30
3.2 Definicja dużych modeli językowych 32
**4. Agent LLM** **36**
4.1 Definicja agenta llm 37
4.2 Architektura agentów LLM 37
4.3 Porównanie: Agent LLM vs. Agent motywowany 40
**5. Implementacja agenta LLM** **42**
5.1 Graf środowiska 42
5.2 Konfiguracja środowiska w Google Colab 46
5.3 Kod agenta 47
5.4 Implementacja środowiska agenta 55
5.5 Pętla agenta 63
5.6 Analiza wyników symulacji 67
5.6.1 Analiza dynamiki baterii, nasłonecznienia i środków finansowych 68
5.6.2 Analiza dynamiki zasobów wodnych w regulacji wilgotności gleby 71
5.6.3 Porównanie z jaką częstotliwością były wybierane akcje 74
**6. Podsumowanie** **81**
**Bibliografia** **83**

# Wstęp

W ostatnich latach obserwujemy dynamiczny rozwój sztucznej inteligencji, w tym systemów opartych na uczeniu ze wzmocnieniem oraz dużych modelach językowych. Szczególne zainteresowanie budzą samodzielnie działające systemy, zdolne do podejmowania decyzji w złożonych i dynamicznych środowiskach, które potrafią adaptować swoje działania w odpowiedzi na zmieniające się warunki oraz własne stany zasobów.

Jednym z inspirujących modeli teoretycznych w tym kontekście jest koncepcja agentów motywowanych (motivated agents), w których działania wynikają z dążenia do redukcji bólu, takiego jak niski poziom energii czy brak zasobów niezbędnych do funkcjonowania. W odróżnieniu od klasycznego RL, gdzie celem jest maksymalizacja nagrody zewnętrznej, agent motywowany priorytetyzuje działania zgodnie z własnymi potrzebami i reaguje przede wszystkim na stany krytyczne.

Niniejsza praca koncentruje się na opracowaniu i analizie agenta, który wykorzystuje Duży Model Językowy jako komponent decyzyjny w symulowanym środowisku ogrodowym. W ramach eksperymentu zaprojektowano dwóch agentów operujących w identycznym środowisku, których zadaniem było podejmowanie decyzji w odpowiedzi na krytyczne stany zasobów w trakcie 1000 iteracji. W eksperymencie uwzględniono dwa stany krytyczne – tzw. „bóle prymitywne”: niski poziom naładowania baterii robota oraz niski poziom wilgotności gleby. Decyzje agentów były generowane na podstawie promptów przekazywanych do LLM oraz bieżących obserwacji, a następnie analizowane pod kątem ich zdolności do adaptacyjnego reagowania zarówno na stany krytyczne, jak i na bóle abstrakcyjne wynikające ze zmieniających się warunków, co umożliwiało utrzymywanie ciągłości działania systemu.

W eksperymencie wykorzystano dwa warianty promptowania. W pierwszym wariancie agent opierał się na podstawowym opisie stanu środowiska i własnej interpretacji zasobów. W drugim wariancie prompt został wzbogacony o szczegółowe informacje dotyczące poziomu dostępności zasobów oraz sugestie rekomendowanych działań. Takie podejście pozwalało zbadać wpływ jakości i szczegółowości promptu na decyzje agenta oraz jego zdolność do generowania zachowań przypominających mechanizmy motywacyjne.

Celem pracy było ocenienie potencjału dużego modelu językowego jako komponentu decyzyjnego w autonomicznym systemie, w tym jego zdolności do generowania zachowań zbliżonych do agentów motywowanych i priorytetyzowania działań w odpowiedzi na sygnały krytyczne. Wyniki eksperymentów wykazały, że oba warianty agentów reagowały na stany krytyczne zasobów w sposób zbliżony do zachowań agentów motywowanych. Agent z podstawowym promptem charakteryzował się większą elastycznością i lepszą koordynacją działań, podczas gdy agent ze zmodfikowanym promptem również reagował zgodnie z mechanizmem „progu bólu", choć w nieco bardziej ograniczony sposób.

Przeprowadzone badania pozwalają wnioskować, że integracja LLM z samodzielnie działającymi systemami może wspierać podejmowanie decyzji adaptacyjnych, przypominających działanie systemów motywowanych. Wyniki te stanowią punkt wyjścia do dalszych badań nad wykorzystaniem modeli językowych w autonomicznych systemach decyzyjnych i eksploracją ich potencjału w kontekście zarządzania zasobami w środowiskach dynamicznych.

# 1. Agenci i uczenie ze wzmocnieniem

W niniejszym rozdziale przedstawione zostaną podstawowe pojęcia związane z konstruowaniem autonomicznych agentów uczących się. Szczególny nacisk położono na omówienie klasycznego paradygmatu uczenia ze wzmocnieniem, który stanowi fundament dla rozwoju bardziej zaawansowanych koncepcji sterowania i podejmowania decyzji. Rozdział obejmuje definicję agenta, opis struktury środowiska, a także rolę funkcji nagrody i celu działania. Następnie omówione zostaną główne komponenty procesu uczenia ze wzmocnieniem wraz z przykładem klasycznego algorytmu Q-learning, który przez wiele lat stanowił podstawowy punkt odniesienia w badaniach nad agentami uczącymi się. W dalszej części zaprezentowane zostaną ograniczenia tradycyjnych podejść oraz przedstawiona koncepcja agenta motywowanego, stanowiącego alternatywę dla klasycznych rozwiązań.

## 1.1 Pojęcie agenta w informatyce

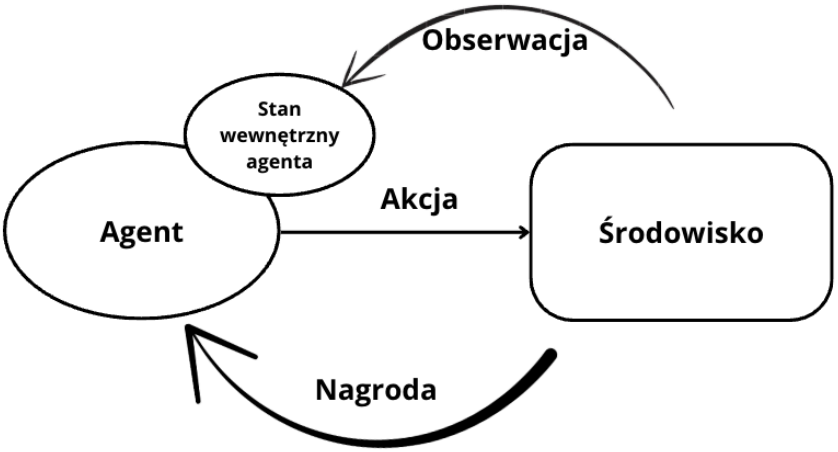
W informatyce oraz w dziedzinie sztucznej inteligencji termin *agent* odnosi się do jednostki obliczeniowej lub programowej, która potrafi samodzielnie odbierać bodźce ze środowiska i podejmować działania w celu osiągnięcia określonych celów. Russell i Norvig definiują agenta jako „cokolwiek, co może odbierać swoje środowisko poprzez sensory i działać na to środowisko poprzez efektory" [40]. Agent może być zarówno fizycznym bytem, jak np. robot, jak i bytami niematerialnymi – np. programem komputerowym wykonującym zadania w środowisku wirtualnym.

Agent podejmuje decyzje na podstawie strategii działania (ang. policy) – strategii przypisującej akcję do każdego możliwego stanu. W kontekście uczenia maszynowego istotne jest, że strategia ta może być modyfikowana na podstawie doświadczenia, co pozwala agentowi adaptować się do zmieniającego się środowiska.

## 1.2 Uczenie ze wzmocnieniem

Uczenie ze wzmocnieniem (ang. *reinforcement learning*, RL) to podejście, w którym agent nabywa wiedzę o sposobie działania w danym środowisku poprzez bezpośrednią interakcję z jego elementami. Środowisko dostarcza agentowi informacji o swoim aktualnym stanie, na podstawie których agent podejmuje decyzje o wykonaniu określonych akcji. Jeżeli

podjęta akcja skutkuje nagrodą, agent modyfikuje prawdopodobieństwo powtórzenia tego działania w przyszłości – nagroda dodatnia zwiększa szansę na wybór tej samej akcji w analogicznej sytuacji, natomiast nagroda ujemna ją zmniejsza. Proces ten jest iteracyjny i prowadzi do systematycznego doskonalenia zachowania agenta, który na podstawie obserwacji skutków swoich działań uczy się wybierać te akcje, które maksymalizują oczekiwaną sumę sygnałów nagrody (Rys. 1) [44].




Rysunek 1. Schemat interakcji agenta bazującego na uczeniu ze wzmocnieniem ze środowiskiem.

### 1.2.1 Główne komponenty uczenia ze wzmocnieniem

Klasyczny model RL opiera się na trójkącie: agent–środowisko–cel (Rys. 1), ale można wyróżnić cztery dodatkowe kluczowe elementy, które nadają strukturę i funkcjonalność całemu procesowi:

- Strategia działania: To sposób wyboru akcji w danym stanie środowiska. Określa regułę, według której agent decyduje się na konkretne działanie – przykładowo w Q-learningu z podejściem epsilon-greedy, czyli wybierania akcji o największej wartości Q (wartość reprezentuje przewidywany zysk związany z podjęciem określonej akcji w danym stanie środowiska) z dużym prawdopodobieństwem oraz akcji losowej z małym prawdopodobieństwem ε, agent może jednocześnie wykorzystywać dotychczasową wiedzę i eksplorować nowe możliwości [44].
- Sygnał nagrody: Określa cel agenta w systemie RL. Każda wykonana akcja może skutkować otrzymaniem (lub brakiem) nagrody. Głównym celem agenta jest maksymalizacja sumy otrzymanych nagród w dłuższym okresie [44].

- Funkcja wartości stanu (ang. state value function): Odróżnia się od sygnału nagrody tym, że opisuje wartość danego stanu w kontekście wszystkich przyszłych stanów, które mogą z niego wyniknąć. O ile nagroda może być natychmiastowa, funkcja wartości uwzględnia długofalowe konsekwencje [44]. Określa ona oczekiwaną sumę zdyskontowanych nagród (czyli przyszłych nagród pomniejszonych o współczynnik $\gamma$, co sprawia, że im później nagroda zostanie otrzymana, tym mniejszy ma wpływ na wartość stanu, przypisaną stanowi $s$, jakie agent może uzyskać, zaczynając od stanu $s$ i podążając dalej zgodnie ze strategią $\pi$:

$$V^{\pi}(s) = E_{\pi}[\sum_{t=0}^{\infty} \gamma^{t} R_{t+1} | S_0 = s]$$

Równanie 1. Funkcja wartości

gdzie:

- $s$ – stan początkowy,
- $\pi$ – strategia agenta,
- $R_{t+1}$ – nagroda otrzymana po wykonaniu akcji w kroku t,
- $\gamma \in [0,1]$ – współczynnik dyskontujący (określa wagę przyszłych nagród),
- $E_{\pi}$ - wartość oczekiwana względem wszystkich możliwych przebiegów procesu przy założeniu, że agent działa zgodnie ze strategią $\pi$

- Model środowiska (model): Jest to opcjonalny składnik systemu RL, który umożliwia agentowi przewidywanie konsekwencji potencjalnych działań. Dzięki modelowi agent może symulować możliwe stany środowiska i oceniać, które z nich prowadzą do najlepszych efektów [44].

### 1.3 Środowisko i cel działania agenta

Środowisko w RL to wszystko, co znajduje się poza agentem – świat, z którym agent wchodzi w interakcje. Agent obserwuje stan środowiska, natomiast środowisko określa skutki podejmowanych przez niego działań. Rola środowiska jest kluczowa – definiuje kontekst, w którym odbywa się uczenie i testowanie strategii [41].

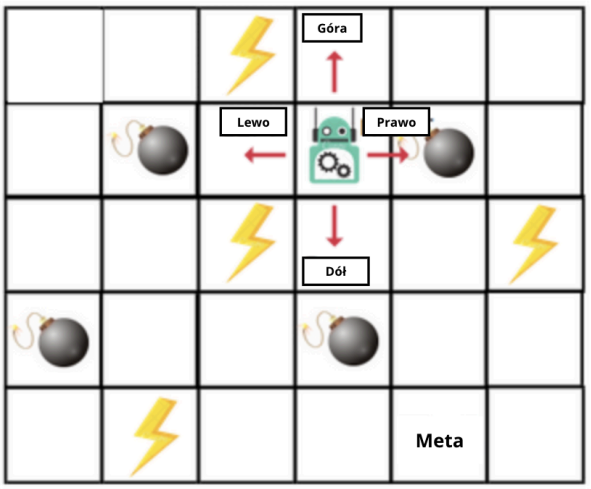
Celem agenta w RL jest maksymalizacja skumulowanej nagrody w długim horyzoncie czasowym. W tym celu agent uczy się funkcji wartości stanu (lub wartości pary stan–akcja), która szacuje oczekiwaną przyszłą nagrodę w zależności od obecnego stanu i decyzji [42].

## 1.4 Q-learning – klasyczny algorytm RL

Q-learning to jedno z najbardziej rozpowszechnionych podejść w kategorii bezmodelowego uczenia ze wzmocnieniem (ang. model-free reinforcement learning), które umożliwia agentowi zdobywanie wiedzy o środowisku poprzez bezpośrednie interakcje z nim – bez potrzeby budowania jego wewnętrznej reprezentacji. Proces ten opiera się na eksperymentowaniu: agent podejmuje działania w różnych stanach środowiska, obserwuje ich konsekwencje, otrzymuje nagrody (lub kary) i w oparciu o nie uczy się optymalnej strategii postępowania. W przeciwieństwie do metod modelowych, takich jak dynamiczne programowanie (np. algorytmy *Value Iteration* czy *Policy Iteration*)[1], które wymagają znajomości dynamiki środowiska, Q-learning nie zakłada dostępu do rozkładu nagród [67].

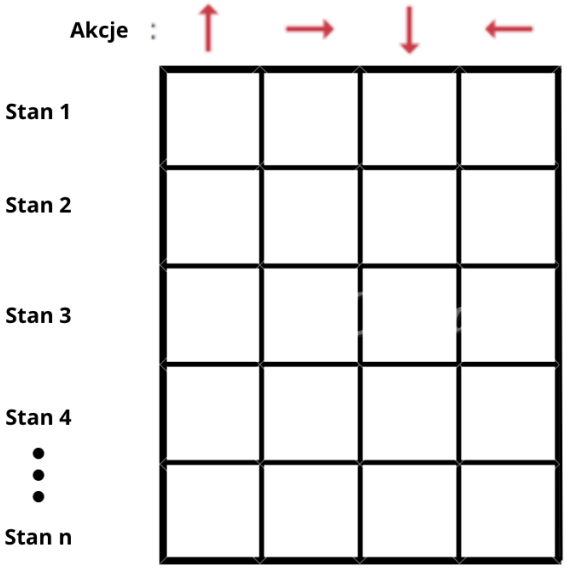
Aby zilustrować działanie algorytmu Q-learning, rozważmy środowisko labiryntu, po którym porusza się robot. Celem robota jest dotarcie do punktu końcowego (mety) w możliwie najkrótszym czasie, unikając przy tym pól minowych. Jednocześnie na jego drodze mogą pojawić się bonusowe pola energetyczne, za które otrzymuje dodatkowe punkty. Każdy ruch wiąże się z niewielką stratą punktową (np. -1), co motywuje agenta do szybkiego ukończenia zadania. Wejście na pole minowe skutkuje natychmiastowym zakończeniem epizodu i dużą stratą punktów (np. -100), natomiast osiągnięcie celu przynosi znaczną nagrodę (np. +100). Z kolei zebranie energii może przynieść niewielką nagrodę dodatnią (np. +1) (Rys. 2).

[1] Programowanie dynamiczne – technika projektowania algorytmów przeznaczonych do rozwiązywania zagadnień optymalizacyjnych wynaleziona przez amerykańskiego matematyka Richarda Bellmana [77].




Rysunek 2. Przykładowe środowisko dla agenta uczącego się metodą Q-learning.

W tak skonstruowanym środowisku agent musi nauczyć się podejmować decyzje, które z jednej strony minimalizują ryzyko (np. unikanie min), a z drugiej maksymalizują długoterminową korzyść (np. dotarcie do celu z premią punktową). Kluczowym elementem tego procesu jest Q-tablica (ang. Q-table) – struktura danych w formie dwuwymiarowej macierzy, w której przechowywane są tzw. wartości Q, czyli oszacowania oczekiwanej wartości przyszłej nagrody dla każdej pary stan–akcja (Rys. 3).




Rysunek 3. Reprezentacja wartości Q-tabeli w środowisku siatki.

Q-tablica pełni rolę pamięci, w której agent zapisuje, jak opłacalne było wykonanie konkretnego działania w danym stanie na podstawie dotychczasowych doświadczeń. Wiersze tablicy reprezentują wszystkie możliwe stany środowiska, natomiast kolumny odpowiadają dostępnym akcjom (np. ruch w górę, dół, w lewo, w prawo) (Rys. 3). Każda komórka tablicy zawiera wartość *Q(s, a)*, czyli szacunkową miarę użyteczności wykonania akcji *a* w stanie *s*. Wartości te są aktualizowane iteracyjnie na podstawie algorytmu uczenia – po każdym kroku agent koryguje swoje oszacowania, zgodnie ze wzorem aktualizacji Q-learningu:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[ r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

Równanie 2. Q-learning update rule.

gdzie:

- *Q(s,a)* – wartość Q dla stanu *s* i akcji *a*,
- *α*– współczynnik uczenia (learning rate),
- *r* – nagroda otrzymana po wykonaniu akcji *a* w stanie *s*,
- *γ* – współczynnik dyskontujący przyszłe nagrody (discount factor)
- $\max_{a'} Q(s',a')$ - maksymalna wartość Q osiągalna w nowym stanie *s′*.

Po wielu cyklach interakcji ze środowiskiem agent – dysponując wyuczonymi wartościami Q, które pozwalają zbudować „dobrą” tablicę wartości Q – może w sposób efektywny poruszać się po środowisku, wybierając działania, które prowadzą go do celu w sposób umożliwiający maksymalizację nagrody.

Choć Q-learning opiera się na stosunkowo prostej idei tablicy odwzorowującej wartość działań, jego zastosowania obejmują szeroki zakres problemów decyzyjnych, od gier planszowych po sterowanie robotami mobilnymi.

### 1.5 Wady klasycznych agentów RL

Tradycyjne podejścia do uczenia ze wzmocnieniem mają wiele ograniczeń, które utrudniają ich skuteczne zastosowanie w bardziej realistycznych i złożonych środowiskach.

Jednym z najczęściej spotykanych problemów jest niewielka liczba nagród dostępnych dla agenta podczas eksploracji. W wielu przypadkach wartości zwrotne pojawiają się dopiero po wykonaniu wielu kroków, co znacznie wydłuża czas potrzebny na skuteczne uczenie się właściwych zachowań i może prowadzić do braku postępu w treningu [55].

Drugim istotnym ograniczeniem jest brak motywacji wewnętrznej – agent uczy się tylko wtedy, gdy otrzymuje sygnały z otoczenia w postaci nagród. W sytuacjach, gdy środowisko nie dostarcza wyraźnej informacji zwrotnej, agent nie jest skłonny eksplorować ani podejmować działań mogących prowadzić do odkrycia nowych strategii [56].

Kolejną trudnością jest eksploracja w dużych przestrzeniach stanów. W miarę wzrostu złożoności środowiska klasyczne strategie eksploracyjne, stają się mało skuteczne, ponieważ losowe wybory nie gwarantują wystarczającej różnorodności zachowań ani efektywnego zbierania informacji [55].

Co więcej, klasyczne podejścia w uczeniu ze wzmocnieniem opierają się wyłącznie na liczbowych reprezentacjach wartości i nagród, przez co agent nie ma świadomości znaczenia wykonywanych działań czy obserwowanych stanów. Zachowanie agenta jest efektem optymalizacji wartości oczekiwanej, a nie zrozumienia kontekstu czy celu działania [57]. Aby przeciwdziałać tym ograniczeniom, opracowano bardziej zaawansowane koncepcje, takie jak hierarchiczne uczenie ze wzmocnieniem (ang. hierarchical RL). W ramach tych metod agent jest w stanie organizować swoje działania w postaci planów złożonych z mniejszych celów – na przykład: „zdobądź klucz”, „otwórz drzwi”, „przejdź do następnego pomieszczenia” [58]. Taka struktura pozwala agentowi lepiej zarządzać eksploracją i rozwiązywać problemy, w których występują rzadkie nagrody.

Hierarchiczne RL umożliwia uczenie się na różnych poziomach abstrakcji, gdzie wyższe poziomy ustalają ogólne cele, a niższe realizują konkretne akcje potrzebne do ich osiągnięcia. Przykładem mogą być architektury takie jak Feudal RL czy Option-Critic, które pozwalają agentowi automatycznie formułować podcele i optymalizować strategie ich

realizacji [59, 60]. Dzięki temu systemy te lepiej radzą sobie ze złożonymi środowiskami, wymagającymi planowania oraz adaptacji do zmiennych warunków.

### 1.6 Koncepcja agenta motywowanego

Aby pokonać niektóre z ograniczeń klasycznego RL (m.in. silną zależność od zewnętrznych nagród oraz trudności w eksploracji), wprowadzono koncepcję agenta motywowanego [1]. Taki agent posiada systemy motywacji wewnętrznej – przede wszystkim wewnętrzne cele (potrzeby, które musi zaspokoić), a także potrzebę eksploracji i ciekawość – które działają niezależnie od zewnętrznych nagród. Inspiracją dla tego podejścia są mechanizmy biologiczne i psychologiczne, znane z badań nad motywacją u ludzi i zwierząt [43]. Agent motywowany może skuteczniej uczyć się w środowiskach pozbawionych nagród, ponieważ potrafi nadawać wartość samemu procesowi eksploracji. W rezultacie działa on bardziej elastycznie i adaptacyjnie, nawet w nieznanych lub zmiennych warunkach. Definicja agenta motywowanego oraz jego działanie będzie dokładniej opisane w rozdziale 2.

# 2. Uczenie motywowane i agenci motywowani

Agent motywowany oraz uczenie motywowane to ściśle powiązane ze sobą koncepcje w dziedzinie uczenia maszynowego i sztucznej inteligencji, zaprojektowane dla autonomicznych agentów wcielonych (ang. *embodied agents*)[2][2]. W przeciwieństwie do tradycyjnych podejść, które opierają się głównie na maksymalizacji zewnętrznie definiowanej nagrody. Paradygmat uczenia motywowanego skupia się na wewnętrznych stanach agenta oraz mechanizmach minimalizacji sygnałów negatywnych — tzw. bólu. Kluczowe dla tych systemów jest samodzielne tworzenie i zarządzanie celami, dynamiczne reagowanie na środowisko[3] oraz uczenie się w oparciu o potrzeby pierwotne i abstrakcyjne.

Celem niniejszego rozdziału jest przedstawienie podstawowych założeń i komponentów systemów motywowanych, ze szczególnym uwzględnieniem ich architektury poznawczej oraz różnic względem klasycznego uczenia ze wzmocnieniem. W kolejnych podrozdziałach omówiona zostanie definicja oraz charakterystyka agenta motywowanego, teoretyczne fundamenty uczenia motywowanego, kluczowe różnice pomiędzy agentem motywowanym a uczeniem ze wzmocnieniem, a także znaczenie sygnałów bólu i potrzeb pierwotnych jako podstawowych mechanizmów napędzających zachowanie agenta.

## 2.1 Definicja i charakterystyka agenta motywowanego

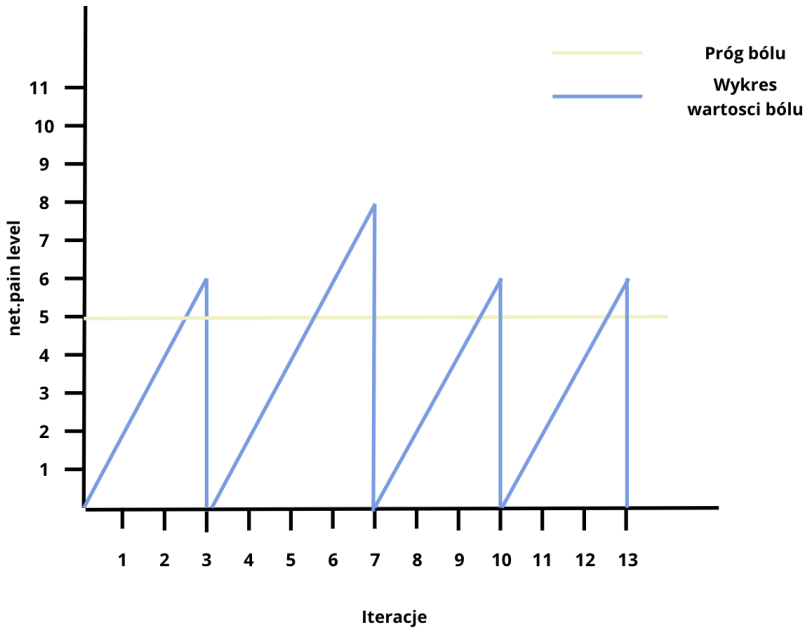
Agent motywowany, czyli *motivated agent*, to rodzaj systemu sztucznej inteligencji, czyli programu komputerowego zdolnego do „uczenia się” i podejmowania decyzji, który został zaprojektowany tak, by działać w zmiennym i często nieprzyjaznym otoczeniu, czyli w środowisku, gdzie warunki często się zmieniają i mogą być niekorzystne [2].

Najważniejszą cechą takiego agenta jest jego zdolność do autonomicznego działania, czyli samodzielnego podejmowania decyzji, bez ciągłego nadzoru człowieka. Agent

[2] Agent wcielony to system sztucznej inteligencji, który posiada „ciało” – fizyczne lub wirtualne – umożliwiające mu percepcję i działanie w określonym środowisku. Wyposażony w sensory i efektory odbiera sygnały z otoczenia, przetwarza je, a następnie podejmuje decyzje prowadzące do realnych zmian w środowisku. Kluczowe jest to, że proces uczenia oraz adaptacji agenta wcielonego wynika bezpośrednio z interakcji z otoczeniem, a nie z abstrakcyjnych obliczeń oderwanych od kontekstu działania. [75]

[3] Przez dynamiczne reagowanie na środowisko rozumie się wybór stosownej akcji w zależności od aktualnego stanu środowiska, który może ulegać zmianom niezależnym od agenta, np. gdy na drodze robota pojawi się przeszkoda, agent decyduje się ją ominąć [67].

motywowany podejmuje decyzje kierując się swoimi wewnętrznymi motywacjami (potrzebami), które bezpośrednio wynikają z odczuwanego przez agenta bólu. Celem agenta jest wybranie takiej akcji na środowisku, która spowoduje zmniejszenie odczuwanego bólu poniżej wartości progowej [2].




Rysunek 4. Zmiany poziomu bólu w zależności od iteracji i dostępności zasobów

Ból, rozumiany tutaj szeroko jako każde nieprzyjemne doświadczenie – fizyczne, emocjonalne lub funkcjonalne – narasta w sposób liniowy ze stałą wartością przyrostu na każdą iterację, co odzwierciedla naturalny proces eskalacji problemu w braku interwencji (Rys. 4). W sytuacji, gdy agent zidentyfikuje i zastosuje odpowiednią kombinację zasobu oraz akcji, udaje mu się skutecznie zminimalizować poziom bólu, co jest widoczne jako gwałtowny spadek na wykresie. Proces uczenia się agenta obejmuje nabycie umiejętności redukcji bólu, co nastąpiło w iteracji 7; od tego momentu agent konsekwentnie korzysta z tej wiedzy, optymalizując swoje działania w celu utrzymania bólu na niskim poziomie. W okresach, gdy ból osiąga wyższe wartości niż oczekiwane (np. w iteracjach poprzedzających dostępność zasobu), wynika to z braku odpowiedniego zasobu do redukcji, co powoduje dalszy wzrost bólu aż do momentu jego pojawienia się i wykorzystania. Zawsze po przekroczeniu ustalonego progu bólu agent inicjuje mechanizmy redukcyjne, dążąc do przywrócenia równowagi. Jeśli jednak zasób niezbędny do redukcji pozostaje niedostępny, poziom bólu kontynuuje wzrost, aż do czasu, gdy zasób stanie się dostępny – co może być

efektem samoistnych zmian w środowisku lub celowych działań podejmowanych przez samego agenta (Rys. 4).

Agent motywowany może wytworzyć wiele sygnałów bólu, które można podzielić na dwa główne typy:

- Ból pierwotny, nazywany też prymitywnym, to podstawowy rodzaj bólu – oznacza z góry zaprogramowane potrzeby agenta, takie jak potrzeba „jedzenia”- dostarczania energii, zaspokajanie „pragnienia”, lub utrzymywanie komfortu termicznego”, czyli unikania zimna lub gorąca. Te sygnały są ustalone przez projektanta systemu i stanowią podstawę jego motywacji [2].

- Ból abstrakcyjny to bardziej złożona forma bólu, która nie wynika bezpośrednio z niezaspokojenia potrzeb podstawowych, lecz z procesów poznawczych agenta, w tym nabywania nowych umiejętności, sposobów rozumienia świata i wiedzy o środowisku. W takim przypadku sam agent tworzy potrzebę posiadania określonego zasobu (osiągnięcia pewnego stanu środowiska), mimo że nie jest on ściśle związany z celem istnienia agenta. Przykładem może być sytuacja, w której brak dostępu do zasobu (np. pożywienia) potrzebnego do zaspokojenia pierwotnej potrzeby (np. głodu) generuje wewnętrzny sygnał braku pożywienia – ból abstrakcyjny – mimo braku jeszcze odczuwania bólu pierwotnego (głodu). Ten nowy bodziec uruchamia eksplorację środowiska przez agenta w celu uzupełnienia zasobu pożywienie, prowadząc do odkrycia związku między obiektami (np. lodówka z pożywieniem) a akcjami (np. kupno pożywienia przy użyciu pieniędzy). Brak możliwości zaspokojenia tej wymyślonej potrzeby (posiadania pożywienia w lodówce) prowadzi do odczuwania bólu abstrakcyjnego, który ma źródło w niedoborze zasobu potrzebnego agentowi do redukcji bólu prymitywnego. Podobnie, gdy agentowi będzie brakować pieniędzy, będzie on odczuwał ból wynikający z braku środków, ponieważ uniemożliwia to zaspokojenie potrzeby abstrakcyjnej, tj. posiadania jedzenia w lodówce. W ten sposób agent może budować hierarchię bólu, czyli uporządkowane struktury bardziej złożonych potrzeb, w których sam wytwarza

potencjalne źródła cierpienia [2, 11]. Przykład takiej hierarchii przedstawiono na rysunku 5.




Rysunek 5. Hierarchia bólu: od potrzeby podstawowej do bólu abstrakcyjnego.

W klasycznym uczeniu ze wzmocnieniem, czyli reinforcement learning (RL), celem jest zdobywanie punktów nagrody. W przypadku motivated learning (ML), czyli uczenia motywowanego, celem jest zmniejszenie wartości bólu, a nie maksymalizacja nagrody. Dzięki temu system jest bardziej stabilny i kończy swoje działanie, gdy potrzeba zostaje zaspokojona [2].

Innym ważnym elementem działania agenta motywowanego jest zdolność do samodzielnego wybierania celów. Cele te powstają w wyniku „rywalizacji" pomiędzy różnymi sygnałami bólu – agent wybiera ten, który w danym momencie jest najsilniejszy, czyli odpowiada najbardziej palącej potrzebie [2].

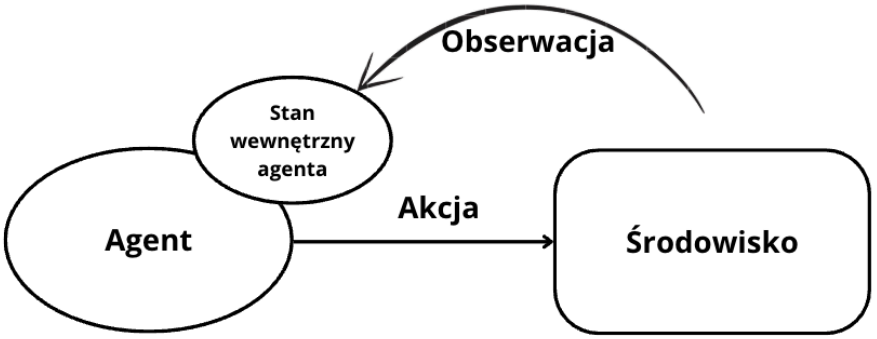
Kolejnym istotnym mechanizmem, który może wspierać rozwój agenta, jest ciekawość (ang. artificial curiosity). Mechanizm ten może być wprowadzony do agenta, ale nie jest aktywny domyślnie – jego uruchomienie zależy od decyzji projektanta systemu. Gdy agent jest wyposażony w ciekawość, działa ona tylko wtedy, gdy żaden z sygnałów bólu nie

przekracza ustalonego progu – czyli gdy agent nie „cierpi” z żadnego powodu. Wówczas może zająć się eksploracją, czyli poszukiwaniem nowych możliwości działania i odkrywaniem skutków własnych akcji w środowisku. Jednak nawet wtedy priorytetem agenta pozostaje zawsze redukcja bólu; ciekawość pełni funkcję narzędzia pomocniczego [4].

### 2.2 Model i architektura agenta motywowanego

Agent motywowany działa w oparciu o cykliczny proces interakcji ze środowiskiem, w którym kluczową rolę odgrywają obserwacja, podejmowanie decyzji oraz wykonywanie akcji – analogicznie jak w klasycznym podejściu RL. Działanie to rozpoczyna się od obserwacji środowiska, która pozwala agentowi zebrać informacje o aktualnym stanie otoczenia. Te informacje, wraz z aktualnym stanem wewnętrznym agenta – odzwierciedlającym jego potrzeby, motywacje i wcześniejsze doświadczenia – wpływają na wybór odpowiedniej akcji. Stan wewnętrzny agenta jest strukturą wewnętrzną, która zawiera dane dotyczące jego aktualnych motywacji (wewnętrznych dążeń ukierunkowujących wybór akcji) oraz „odczuwanego bólu” – rozumianego jako miara niezaspokojenia potrzeby, która narasta w czasie, ale osiąga pewien górny pułap, po którego przekroczeniu nie zwiększa się już jej intensywność. Na tej podstawie agent podejmuje decyzję i wykonuje określoną akcję, czyli działanie mające na celu zmianę sytuacji w środowisku w taki sposób, by zmniejszyć wewnętrzny dyskomfort. Akcja ta wpływa na środowisko, powodując jego reakcję. Środowisko zmienia swój stan w odpowiedzi na działanie agenta, co następnie ponownie podlega obserwacji. Informacje uzyskane w wyniku tej obserwacji trafiają z powrotem do agenta, aktualizując jego stan wewnętrzny i przygotowując go do kolejnego cyklu decyzyjnego. Taka struktura działania pozwala agentowi na ciągłe uczenie się i dostosowywanie do zmiennych warunków zewnętrznych. Dzięki powtarzającemu się cyklowi obserwacji, reakcji i adaptacji, agent jest w stanie rozwijać strategie działania ukierunkowane na minimalizację bólu i skuteczne zaspokajanie potrzeb (Rys. 6). Cechą charakterystyczną tego modelu jest to, że proces uczenia polega na ocenie, czy dane działanie przyczyniło się do minimalizacji 'bólu', a następnie na aktualizacji wag połączeń na podstawie tej informacji. Jeśli dana para akcja–zasób ma największą wartość, agent wzmacnia tę strategię, zwiększając prawdopodobieństwo jej ponownego użycia. W przypadku rozbieżności aktualizuje swoje wewnętrzne reprezentacje, korygując wartości przypisane do poszczególnych działań i stanów. Dzięki temu kolejne decyzje są

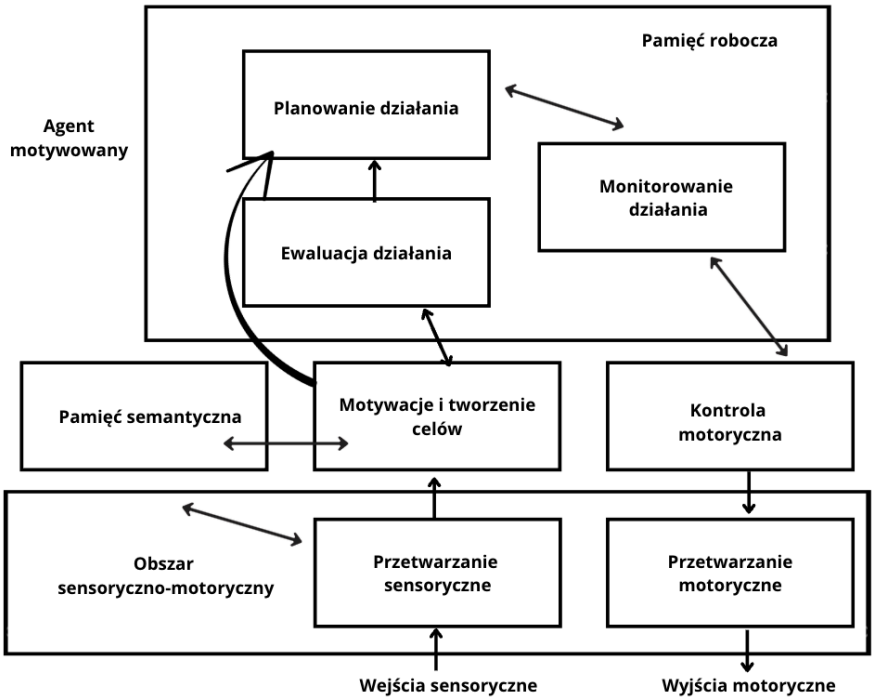
podejmowane w oparciu o coraz precyzyjniejsze oszacowania skuteczności działań. W praktyce agent motywowany uczy się więc poprzez iteracyjne dostosowywanie wag i priorytetów w swoim stanie wewnętrznym, gdzie priorytety oznaczają względną ważność przypisywaną różnym potrzebom lub celom, decydującą o tym, które z nich są realizowane w pierwszej kolejności. Takie mechanizmy umożliwiają agentowi lepsze przewidywanie konsekwencji, skuteczniejsze reagowanie na zmienne warunki i stopniowe budowanie hierarchii strategii [67].




Rysunek 6. Schemat interakcji agenta motywowanego ze środowiskiem - w przeciwieństwie do agenta bazującego na uczeniu ze wzmocnieniem środowisko nie przekazuje informacji o nagrodzie.

Agent motywowany przetwarza dane sensoryczne, odbierane z otoczenia za pomocą „zmysłów" technicznych, takich jak kamery, czujniki czy mikrofony. Dane te trafiają najpierw do modułu przetwarzania sensorycznego, czyli elementu odpowiedzialnego za analizę i interpretację sygnałów wejściowych pochodzących ze środowiska (Rys. 7). Następnie przetworzone informacje są przekazywane do obszaru sensomotorycznego, czyli części systemu odpowiadającej za integrację wrażeń zmysłowych z planowaniem ruchu, oraz do systemu motywacyjnego, czyli struktury tworzącej wewnętrzne potrzeby i cele działania agenta. Cele, czyli określone stany, do których dąży agent — rozumiane jako sytuacje, w których zostaje zredukowany ból prymitywny lub abstrakcyjny poprzez wykonanie odpowiedniej akcji na środowisku i wybranym zasobie — są ustalane na podstawie trzech źródeł: informacji pochodzących z otoczenia, treści pamięci semantycznej, czyli długoterminowej bazy wiedzy o świecie, oraz pamięci epizodycznej, która gromadzi wcześniejsze doświadczenia agenta i umożliwia wykorzystywanie sprawdzonych działań

w podobnych sytuacjach. Gdy cel zostaje określony, do działania przystępuje moduł planowania, czyli komponent systemu opracowujący kroki potrzebne do jego osiągnięcia. W procesie planowania wykorzystywane są dane zawarte w pamięci roboczej, czyli tymczasowej przestrzeni pamięci przechowującej informacje istotne w danym momencie. Po wykonaniu zaplanowanych działań, następuje ich ewaluacja, czyli ocena skuteczności – proces ten polega na analizie, czy działania przyniosły oczekiwany rezultat. Na tej podstawie agent otrzymuje informację zwrotną, czyli dane o tym, czy jego decyzje były trafne. Równocześnie z ewaluacją działa mechanizm monitorowania, czyli system ciągłej kontroli nad przebiegiem realizowanego zadania. Monitorowanie pozwala agentowi w czasie rzeczywistym dostosowywać swoje zachowanie do zmieniających się warunków i unikać błędów. Kiedy cele są już znane, a plan działania gotowy, aktywowana zostaje kontrola motoryczna, czyli proces przekazywania informacji do systemu wykonawczego. System ten, nazywany przetwarzaniem motorycznym, odpowiada za przygotowanie i uruchomienie odpowiednich reakcji fizycznych, czyli tzw. wyjść motorycznych – może to być np. ruch ramienia robota, wydanie dźwięku lub przesłanie danych. Cały opisany proces opiera się na współpracy trzech głównych grup modułów: poznawczych, czyli odpowiedzialnych za analizę informacji i planowanie, motywacyjnych, czyli tworzących cele i motywacje, oraz wykonawczych, czyli realizujących działania. Dzięki tej współpracy agent motywowany może adaptacyjnie (elastycznie, i celowo), czyli w sposób ukierunkowany na osiągnięcie celu, funkcjonować w dynamicznym, zmiennym środowisku [67].




Rysunek 7. Architektura funkcjonalna agenta motywowanego.

Agent ML oddziałuje z wirtualnym środowiskiem. W implementacji opartej na modelu Motivated Learning zakłada się, że zarówno dane wejściowe (sensoryczne), jak i wyjściowe (motoryczne), mają postać symboliczną, czyli są uproszczonymi reprezentacjami obiektów i akcji, które tworzą interfejs pomiędzy agentem a środowiskiem. System Motivated Learning (ML) to podejście do uczenia maszynowego inspirowane mechanizmami motywacyjnymi oparty jest na sieci neuronowej, w której każdy neuron sensoryczny reprezentuje konkretny obiekt ze środowiska, a każdy neuron motoryczny odpowiada konkretnej akcji, jaką agent może wykonać [18] (Rys.8).

Oprócz neuronów sensorycznych (*S*) i motorycznych (*M*), sieć neuronowa zawiera:

- neurony bólu (*P*), które rejestrują sygnały bólu, czyli informacje o niezaspokojonych potrzebach lub negatywnych stanach agenta,

- neurony celu (*G*), które odpowiadają za zmniejszanie bólu poprzez inicjowanie odpowiednich działań,

- neurony uprzedzeń (*B*), które mogą wpływać na decyzje niezależnie od bieżących bodźców – stanowią one formę dodatkowych, wewnętrznych „nastawień”.

Neurony bólu prymitywnego oraz neurony bólu abstrakcyjnego są połączone z sensorami monitorującymi dostępność zasobów w środowisku. Na podstawie sygnałów z tych sensorów neurony bólu abstrakcyjnego określają poziom odczuwanego bólu – gdy zasobów jest dużo, ból jest niski, natomiast ich niedobór prowadzi do jego wzrostu. W klasycznym uczeniu ze wzmocnieniem architektura agenta nie jest wyposażona w neurony bólu zamiast tego agent uczy się na podstawie sygnałów nagrody i funkcji wartości, natomiast w ML neurony bólu otrzymują pierwotne sygnały – informacje o podstawowych, niezaspokojonych potrzebach – które bezpośrednio modulują ich poziom aktywności. W uczeniu motywowanym abstrakcyjne ośrodki bólu, czyli takie które nie odnoszą się do bezpośrednich, fizycznych stanów agenta (np. głodu, pragnienia czy poziomu energii), są tworzone poprzez mechanizm formułowania celów. Ich aktywacja następuje na podstawie interpretacji danych sensorycznych. Cel w motywowanym uczeniu nie jest bezpośrednio związany z pojedynczą parą obiekt–akcja, lecz z redukcją odczuwanego bólu poprzez zdobycie brakującego zasobu. Bodźcem może być np. brak jedzenia w lodówce, natomiast celem działania jest zdobycie zasobu (pieniędzy), który umożliwia realizację potrzeby, a odpowiadająca mu akcja to kupno jedzenia. W ten sposób akcje nie muszą być skierowane na sam obiekt związany z pierwotnym bodźcem, lecz mogą dotyczyć innych elementów środowiska, które pośrednio prowadzą do zaspokojenia potrzeby [18].

Neurony bólu abstrakcyjnego są połączone z sensorami monitorującymi dostępność zasobów w środowisku. Na podstawie sygnałów z tych sensorów neurony bólu abstrakcyjnego określają poziom odczuwanego bólu – gdy zasobów jest dużo, ból jest niski, natomiast ich niedobór prowadzi do jego wzrostu. Poziom bólu można formalnie zapisać za pomocą funkcji (Równanie 3).

$$P = B * W_{BP}$$

$$G = W_{P_{trig}G} * Cert + UA * W_{UAG}$$

Równanie 3. Funkcja bólu w zależności od poziomu zasobów.

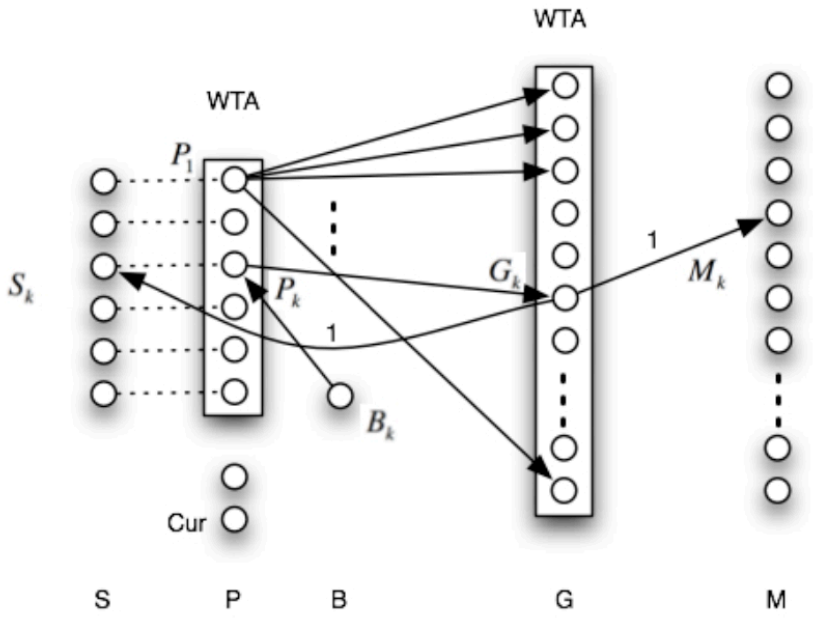
gdzie:

- $P$ - Poziom bólu neuronu abstrakcyjnego,
- $B$ - Intensywność bólu wejściowego – wartość początkowa bólu pochodząca z zewnętrznych bodźców,
- $W_{BP}$ - Waga połączenia pomiędzy bazą bólu a neuronem bólu,
- $G$ - Poziom aktywacji neuronu celu,
- $W_{P_{trig}G}$ - Waga połączenia pomiędzy neuronem bólu ($P$) a neuronem celu ($G$) dla neuronu "*trig*" – waga specyficzna dla neuronu, który wygrał konkurencję,
- *Cert* - Pewność – wskaźnik pewności decyzji lub aktywacji,
- *UA* - Niedostępne zasoby - poziom braku zasobów, który modyfikuje aktywację.
- $W_{UAG}$ - Waga połączenia pomiędzy czynnikiem *UA* a neuronem celu $G$ – określa, jak mocno brak zasobów wpływa na aktywację G.

Architektura agenta obejmuje neurony bólu, neurony decyzyjne oraz połączenia synaptyczne, których wagi są aktualizowane w odpowiedzi na zmiany poziomu bólu. W klasycznym uczeniu ze wzmocnieniem architektura agenta nie jest wyposażona w neurony bólu – zamiast tego agent uczy się na podstawie sygnałów nagrody i funkcji wartości. W modelu neurony bólu otrzymują pierwotne sygnały – informacje o podstawowych, niezaspokojonych potrzebach – które bezpośrednio modulują ich poziom aktywności. W uczeniu motywowanym abstrakcyjne ośrodki bólu, czyli takie, które nie odnoszą się do bezpośrednich, fizycznych stanów, są tworzone poprzez mechanizm formułowania celów. Ich aktywacja następuje na podstawie interpretacji danych sensorycznych. Cel definiowany jest jako zamierzony stan środowiska, w którym ból ulega redukcji, a jego realizacja wymaga wykonania określonej akcji wobec konkretnego obiektu (pary: obiekt–akcja) [18].

Wszystkie neurony bólu początkowo są losowo połączone z neuronami celu, a siły tych połączeń (czyli wagi synaptyczne) są dobierane losowo. Zarówno neurony bólu, jak i neurony celu podlegają mechanizmowi rywalizacji Winner-Take-All (WTA), co oznacza, że tylko jeden neuron może zostać „zwycięzcą” i zdominować inne w danym momencie. Liczba neuronów celu jest równa liczbie możliwych par: bodziec – reakcja [18].

W symbolicznej reprezentacji każdy neuron reprezentuje pojedynczy symbol, którym może być obiekt, ból, cel lub akcja. Na ilustracji przedstawiono symbolicznie strukturę połączeń pomiędzy neuronami sensorycznymi (*S*), bólu (*P*), uprzedzeń (*B*), celów (*G*) i motorycznymi (*M*) (Rys. 8). Na rysunku widoczne są również połączenia jednego, przykładowego neuronu bólu ($P_k$) z jego powiązanymi neuronami sensorycznymi, uprzedzeń, celów oraz motorycznymi [18] (Rys. 8).




Rysunek 8. Połączenia pomiędzy neuronami sensorycznymi, motorycznymi, uprzedzeń, bólu i celów.

## 2.3 Podstawy teoretyczne uczenia motywowanego

Jednym z kluczowych zagadnień związanych z projektowaniem systemów inteligentnych jest stworzenie mechanizmu, który umożliwi realizację wielu funkcji związanych z odbiorem bodźców sensorycznych, przetwarzaniem sygnałów nagradzających i karzących, uczeniem się na ich podstawie, zarządzaniem konkurującymi motywacjami oraz generowaniem nowych, a także sterowaniem działaniami maszyny. Aby maszynę można było uznać za inteligentną, musi ona wykazywać zdolność uczenia się na podstawie interakcji z otoczeniem oraz umiejętność generalizacji zdobytej wiedzy, tak aby skutecznie realizować

swoje cele w zmiennych warunkach. Obejmuje to m.in. rozpoznawanie sytuacji niekorzystnych, takich jak brak zasobów czy niemożność wykonania zaplanowanego działania, oraz dostosowywanie strategii w sposób zwiększający jej sprawczość [11].

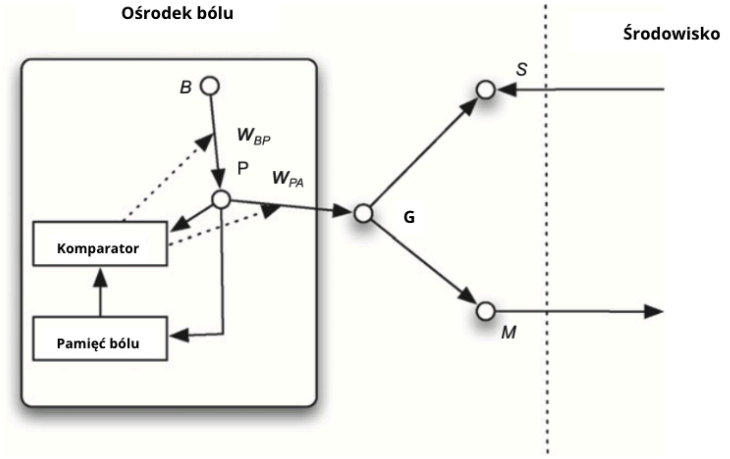
Ból pełni rolę podstawowego sygnału motywacyjnego. Badania neurobiologiczne [39] wskazują, że u ludzi i innych zwierząt jego przetwarzanie angażuje rozbudowaną sieć struktur mózgowych, określaną mianem „matrycy bólu". Ból pełni rolę nie tylko sensoryczną, ale również afektywną (związaną z reakcjami emocjonalnymi) i motywacyjną. W ujęciu systemowym, sieć bólu w systemie uczącym się odpowiada za generowanie motywacji, skupianie uwagi oraz ocenę bodźców zmysłowych i wykonawczych [11].

W przeciwieństwie do klasycznych systemów uczących się w oparciu o maksymalizację nagrody (ang. reward maximization), podejście oparte na unikaniu kary – w szczególności bólu – cechuje się większą stabilnością. Istnieją liczne dowody na to, że systemy oparte wyłącznie na nagrodzie mogą prowadzić do zachowań autodestrukcyjnych, jak np. eksperymentalnie obserwowana stymulacja ośrodków nagrody przez szczury do momentu śmierci z głodu [38]. Redukcja bólu jako strategia motywacyjna nie prowadzi do takich paradoksalnych efektów, ponieważ ma charakter skończony – proces kończy się, gdy ból spada poniżej ustalonego progu [11].

Matematycznie system oparty na minimalizacji bólu można opisać jako proces sekwencyjnej redukcji dominujących sygnałów bólowych. W miarę jak poziom jednego sygnału zostaje zredukowany, system automatycznie przesuwa uwagę na inny, silniejszy w danym momencie sygnał, co pozwala na efektywne zarządzanie wieloma celami jednocześnie [11].

Mechanizm uczenia motywowanego opiera się na rozpoznawaniu bólu oraz systemie jego zapamiętywania. Gdy poziom bólu wzrasta, agent rozpoczyna eksplorację możliwych działań poprzez aktywację neuronów wykonawczych. Proces ten nie jest jednak czysto losowy: w ośrodku bólu sygnały z neuronów bólu (B) oraz neuronów predykcji (P) są zestawiane przez komparator, który porównuje bieżący poziom bólu z informacjami zapisanymi w pamięci bólu. Pamięć ta przechowuje wcześniejsze doświadczenia agenta, czyli informacje o tym, jakie działania prowadziły do redukcji bólu w przeszłości. Wagi $W_{BP}$ i $W_{PA}$ regulują siłę powiązań między neuronami bólu, predykcji i akcji, a ich wartości są modyfikowane w procesie uczenia (Rys. 9). Sygnały bólowe i motoryczne konkurują ze sobą

w trybie „zwycięzca bierze wszystko” (ang. Winner-Take-All), co pozwala na wyłonienie dominującego impulsu motywacyjnego i podjęcie adekwatnej reakcji (M) wobec środowiska (S). Jeżeli wykonane działanie prowadzi do zmniejszenia bólu, odpowiednie połączenia synaptyczne zostają wzmocnione. W efekcie skuteczne reakcje stają się preferowanymi odpowiedziami na konkretne sygnały bólowe. W ten sposób agent stopniowo uczy się powiązań pomiędzy sygnałami sensorycznymi (S), poziomem bólu i skutecznymi działaniami (M), a dzięki pamięci bólu i mechanizmowi porównywania może nie tylko reagować, lecz także uogólniać wcześniejsze doświadczenia na nowe sytuacje [11].




Rysunek 9. Podstawowa jednostka diagnostyczna i edukacyjna dotycząca wykrywania bólu.

W ramach uczenia motywowanego agent nie tylko rejestruje bodźce i działania, lecz stopniowo buduje i udoskonala wewnętrzne reprezentacje sensoryczne (opisy bodźców ze środowiska) oraz motoryczne (opisy własnych możliwych akcji), kierując się przede wszystkim ich rolą w zmniejszaniu odczuwanego bólu.

## 2.4 Różnice między uczeniem motywowanym a klasycznym uczeniem ze wzmocnieniem

Różnice między uczeniem motywowanym (ML) a klasycznym uczeniem ze wzmocnieniem (RL) są fundamentalne i wynikają z odmiennych mechanizmów motywacyjnych oraz celów procesu uczenia się [8].

Różnice między uczeniem motywowanym (ML) a klasycznym RL:

- Źródło motywacji:
  - ML: wewnętrzne – sygnały bólu, dyskomfortu, brak zasobów
  - RL: zewnętrzne – nagrody dostarczane przez środowisko

- Cel uczenia:
  - ML: minimalizacja stanów niepożądanych (ból, brak zasobów)
  - RL: maksymalizacja skumulowanej nagrody

- Rola sygnału:
  - ML: sygnał bólu pełni funkcję nagrody; działanie podejmowane, gdy poziom bólu przekroczy próg
  - RL: nagroda pochodzi z otoczenia; działanie wybierane zgodnie z funkcją wartości

- Aktywacja działań:
  - ML: akcja wykonywana tylko przy znaczącym sygnale bólu
  - RL: akcja wybierana w każdym kroku

Kolejna różnica dotyczy stabilności systemu. W ML minimalizacja bólu prowadzi do jego naturalnego wygaszenia, co sprzyja stabilności i przewidywalności działania [8]. Gdy ból spadnie poniżej ustalonego progu, agent przestaje działać lub przechodzi w stan eksploracji [9]. W RL, z powodu nieograniczonej możliwości maksymalizacji nagrody, systemy mogą wykazywać niestabilność [9]. W zakresie tworzenia celów i autonomii ML wyróżnia się możliwością samodzielnego definiowania celów, których projektant nie musi wcześniej zdefiniować. Agent uczy się ustalać priorytety w zależności od dominujących sygnałów bólu, tworząc hierarchię celów – od pierwotnych po złożone i abstrakcyjne [8]. W RL cele są zazwyczaj zewnętrznie określone i trudno jest zaimplementować w pełni autonomiczne tworzenie celów [9].

Jeśli chodzi o efektywność i szybkość uczenia, agenci ML wykazują znacznie szybsze tempo nauki, szczególnie w złożonych środowiskach – nawet od 10 do 100 razy szybciej niż klasyczne RL [10]. Dzieje się tak, ponieważ działanie jest uruchamiane jedynie w razie

potrzeby [9]. RL, szczególnie w kontekście wartościowania stanów i akcji, jest wolniejszy i bardziej podatny na problemy zbieżności w środowiskach o dużej złożoności [9].

Równie ważnym aspektem jest tzw. problem przypisania kredytu (ang. Credit Assignment Problem), który polega na trudności w określeniu, które z wcześniejszych działań agenta były odpowiedzialne za uzyskanie późniejszej nagrody lub kary. Innymi słowy, gdy nagroda pojawia się z opóźnieniem, agent musi ustalić, którym dokładnie decyzjom lub krokom „przypisać zasługę" za osiągnięty rezultat [55]. W ML występuje mniejsza podatność na ten problem, ponieważ sygnały bólu mają charakter abstrakcyjny i mogą być bezpośrednio powiązane z pierwotnymi potrzebami [9]. W RL przypisanie odpowiedzialności za daną nagrodę do konkretnego działania jest jednak trudne [9]. W kontekście eksploracji środowiska, ML zakłada, że ciekawość uruchamiana jest tylko wtedy, gdy żaden z sygnałów bólu nie przekracza progu działania [1]. W efekcie, eksploracja służy jako wtórna strategia, aktywowana w czasie braku pilnych potrzeb. W RL eksploracja ma często charakter losowy i jest integralną częścią strategii maksymalizacji nagrody [9].

Agenci ML są także lepiej przystosowani do działania w wrogim, dynamicznym środowisku, które generuje ciągłe niedobory i zagrożenia [8]. Dzięki wewnętrznym motywacjom potrafią lepiej zarządzać zasobami i dostosowywać się do zmiennych warunków [11]. RL może mieć trudności w takich sytuacjach, zwłaszcza gdy oszacowanie wartości stanu nie nadąża za zmianami [11].

W przypadku problemów obejmujących wiele celów, ML okazuje się bardziej naturalnie przystosowany do ich realizacji. Poprzez konkurencję między sygnałami bólu agent potrafi selekcjonować cele dynamicznie i priorytetowo [11]. RL natomiast wykazuje ograniczenia przy jednoczesnym uwzględnianiu wielu, często sprzecznych, celów w jednym procesie optymalizacyjnym [12]. Warto wspomnieć o wpływie pamięci semantycznej oraz dużych modeli językowych (LLM). Uczenie motywowane może być wzbogacone o wiedzę kontekstową zapisaną w strukturach takich jak grafy wiedzy (np. ANAKG), co przyspiesza proces uczenia i poprawia zarządzanie zasobami [13].

# 3. Duże modele językowe (LLM) w kontekście sztucznej inteligencji

W ostatnich latach duże modele językowe (LLM, ang. Large Language Models) stały się kluczowym elementem rozwoju współczesnej sztucznej inteligencji. Ich zdolność do generowania, rozumienia i analizowania języka naturalnego sprawia, że coraz częściej pełnią funkcje wykraczające poza typowe przetwarzanie tekstu — wykazują zdolności do rozumowania, planowania i podejmowania decyzji. W niniejszym rozdziale omówione zostaną podstawowe cechy i definicja dużych modeli językowych oraz ich geneza i rozwój, ze szczególnym uwzględnieniem przejścia od klasycznych metod do nowoczesnych modeli opartych na architekturze transformera.

## 3.1 Geneza i rozwój dużych modeli językowych

Rozwój dużych modeli językowych (ang. Large Language Models – LLM) stanowi jedno z najbardziej dynamicznych i przełomowych osiągnięć w dziedzinie sztucznej inteligencji, szczególnie w obszarze przetwarzania języka naturalnego (ang. Natural Language Processing – NLP). Ich historia sięga początków badań nad modelowaniem języka, kiedy to wykorzystywano proste modele statystyczne oparte na n-gramach[4], przewidujące prawdopodobieństwo wystąpienia kolejnych słów w ciągu tekstowym na podstawie wystąpień historycznych [25]. Modele te, choć efektywne w ograniczonym zakresie, miały problemy ze zrozumieniem dłuższych zależności semantycznych i składniowych.

Istotnym etapem w rozwoju modeli językowych było zastosowanie rekurencyjnych sieci neuronowych (RNN) oraz ich ulepszonych wariantów, takich jak LSTM (Long Short-Term Memory) i GRU (Gated Recurrent Unit) [27], które umożliwiły lepsze modelowanie zależności czasowych w sekwencjach tekstowych. Prawdziwy przełom w rozwoju dużych modeli językowych (LLM) nastąpił jednak dopiero wraz z wprowadzeniem architektury transformera [69], która dzięki mechanizmowi uwagi

[4] N-gram to metoda analizy tekstu polegająca na tworzeniu sekwencji słów o długości n. W bigramie (2-gram) drugie słowo jest dobierane na podstawie prawdopodobieństwa jego wystąpienia po pierwszym słowie. W trigramie (3-gram) ostatnie słowo w sekwencji zależy od dwóch poprzednich słów, a w czterogramie (4-gram) od trzech wcześniejszych słów, i tak dalej. [68].

pozwoliła na efektywne przetwarzanie długich fragmentów tekstu i skalowanie modeli do niespotykanych wcześniej rozmiarów.

Prawdziwą rewolucję zapoczątkowało wprowadzenie architektury transformera, zaprezentowanej przez Vaswaniego i współpracowników w pracy *Attention Is All You Need* (2017) [28]. Model ten oparty był na mechanizmie samo-uwagi (self-attention), który umożliwia równoległe przetwarzanie sekwencji oraz efektywne wychwytywanie zależności między słowami niezależnie od ich pozycji w tekście. Transformery zrewolucjonizowały sposób trenowania modeli językowych, znacznie poprawiając efektywność obliczeniową oraz jakość uzyskiwanych reprezentacji językowych. W odróżnieniu od RNN i LSTM, które przetwarzały dane sekwencyjnie krok po kroku, co prowadziło do problemów z długoterminowymi zależnościami i wysokimi kosztami obliczeniowymi, transformery wykorzystywały równoległe przetwarzanie tokenów w obrębie sekwencji dzięki mechanizmowi uwagi, co pozwoliło na szybsze i dokładniejsze modelowanie kontekstu w całym tekście [70].

Na bazie transformera powstały kolejne generacje modeli językowych, takie jak BERT (2018), który wprowadził koncepcję dwukierunkowego kodowania kontekstu, co umożliwiło lepsze rozumienie znaczenia zdań i ich składni [29]. BERT stał się fundamentem dla wielu modeli używanych w zadaniach klasyfikacji, ekstrakcji informacji czy odpowiadania na pytania. Niemal równocześnie OpenAI rozpoczęło rozwój serii GPT (ang. Generative Pre-trained Transformer), wprowadzając modele o coraz większej liczbie parametrów: GPT‑1 (2018) z około 117 milionami parametrów, GPT‑2 (2019) z około 1,5 miliarda parametrów, GPT‑3 (2020) z 175 miliardami parametrów, a wreszcie GPT‑4 (2023), który reprezentuje nową jakość nie tylko w zakresie przetwarzania języka, ale również jego rozumienia i generowania w kontekście multimodalnym [30][31][32][33]. Dokładna liczba parametrów GPT-4 nie została oficjalnie ujawniona, jednak szacunki wskazują na około 1,76 tryliona parametrów, co podkreśla skalę jego zaawansowania [71].

Charakterystyczną cechą nowoczesnych LLM jest ogromna skala. Przykładowo, GPT-3 zawiera 175 miliardów parametrów, a modele takie jak DeepSeek-R1 osiągają nawet 671 miliardów parametrów w architekturze MoE, z czego około 37 miliardów jest aktywnych na token, czyli podstawową jednostkę tekstu (np. słowo, fragment słowa lub znak interpunkcyjny), co pokazuje ich zdolność do efektywnego radzenia sobie z złożonymi problemami językowymi bez konieczności specjalistycznego dostrajania [34]. Równolegle

rozwijane były inne modele o porównywalnym potencjale, takie jak PaLM (Google), LLaMA (Meta) czy Claude (Anthropic), co doprowadziło do gwałtownego wzrostu zainteresowania modelami językowymi jako podstawą dla inteligentnych agentów i systemów autonomicznych [35].

Rozwój LLM nie tylko wpłynął na jakość przetwarzania języka, ale również odmienił sposób podejścia do rozwiązywania problemów ogólnej sztucznej inteligencji. Modele te, odpowiednio trenowane i wykorzystywane, potrafią przejawiać cechy emergentnego rozumowania, czyli zdolności do samodzielnego wyciągania wniosków i rozwiązywania zadań, które nie były jawnie programowane, dzięki złożonym wzorcom powstałym w procesie uczenia [72]. Mogą się uczyć nowych zadań na podstawie pojedynczych przykładów (few-shot learning), a także wspomagać użytkowników w podejmowaniu decyzji w złożonych kontekstach [36]. Ich uniwersalność i skalowalność sprawiają, że są one dziś wykorzystywane w szerokim wachlarzu zastosowań, od chatbotów, przez systemy rekomendacyjne, po wspieranie procesów badawczych i edukacyjnych.

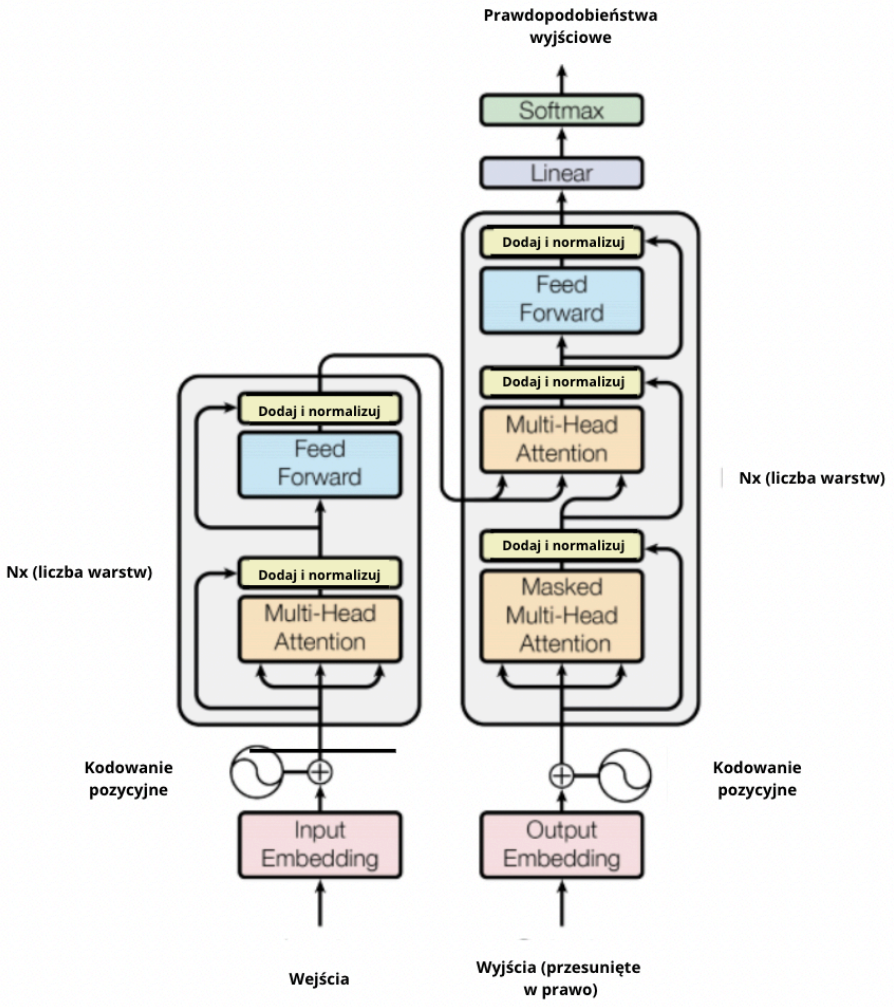
Współcześnie duże modele językowe postrzegane są jako centralny element rozwoju nowej generacji sztucznej inteligencji. Ich ewolucja wciąż trwa, a dynamicznie rozwijające się środowiska badawcze i komercyjne intensywnie pracują nad zwiększeniem ich efektywności, interpretowalności oraz zdolności do działania w rzeczywistych, dynamicznych środowiskach. W perspektywie kolejnych lat możemy spodziewać się dalszego rozszerzania możliwości LLM, zarówno poprzez wzrost skali, jak i integrację z innymi paradygmatami uczenia, jak np. reinforcement learning, co potencjalnie doprowadzi do powstania bardziej zaawansowanych, świadomych kontekstu agentów AI [37].

### 3.2 Definicja dużych modeli językowych

Architektura transformera, będąca fundamentem dużych modeli językowych, umożliwia zaawansowane przetwarzanie języka naturalnego poprzez swoją innowacyjną strukturę. Transformer składa się z dwóch głównych komponentów: enkodera i dekodera, które współpracują ze sobą w procesie przetwarzania języka naturalnego. Enkoder odpowiada za przekształcenie danych wejściowych (np. zdania w języku angielskim) w reprezentację pośrednią, natomiast dekoder generuje dane wyjściowe (np. tłumaczenie w języku francuskim) na podstawie tej reprezentacji. Struktura każdego z komponentów

(zarówno enkodera, jak i dekodera) opiera się na powtarzalnych blokach zawierających kilka kluczowych elementów: multi-head attention, sieci feed-forward, warstwy normalizacji i połączenia rezydualne. Enkoder skupia się na analizie sekwencji wejściowej, natomiast dekoder wykorzystuje również masked attention, która uniemożliwia dostęp do przyszłych tokenów w trakcie generowania tekstu, co pozwala zachować spójność składniową i semantyczną generowanego zdania. Szczegółową architekturę transformera, z uwzględnieniem tych mechanizmów, przedstawiono w kolejnej ilustracji (Rys. 10).

Chociaż zarówno Recurrent Neural Networks (RNN), jak i Transformer pracują na danych sekwencyjnych, różnią się sposobem ich przetwarzania. W RNN kolejne tokeny analizowane są jeden po drugim, a każde wyjście zależy od poprzedniego stanu ukrytego, co powoduje, że obliczenia mają charakter sekwencyjny i ogranicza możliwość ich równoległego wykonywania. Transformer natomiast przetwarza wszystkie tokeny jednocześnie, wykorzystując mechanizm self-attention. Ponieważ mechanizm ten sam w sobie nie uwzględnia kolejności tokenów, stosuje się positional encoding, które dostarcza modelowi informacji o położeniu słów w sekwencji. Dzięki temu Transformer potrafi uchwycić zależności między tokenami niezależnie od ich wzajemnej odległości w tekście [73].




Rysunek 10. Szczegółowa architektura transformera. Po lewej stronie przedstawiono wielowarstwowy enkoder, a po prawej dekoder z mechanizmem zamaskowanej uwagi. Model wykorzystuje kodowanie pozycyjne oraz wielogłową uwagę i warstwy feed-forward w obu komponentach.

Trening LLM przebiega w dwóch etapach: pre-treningu, podczas którego model uczy się przewidywać kolejne słowa na podstawie dużych zbiorów danych, oraz dostrajania (fine-tuning), pozwalającego na dostosowanie modelu do konkretnych zadań, np. asystenta konwersacyjnego. Niektóre modele są dodatkowo trenowane z użyciem informacji zwrotnej od ludzi w procesie Reinforcement Learning from Human Feedback (RLHF) [47].

Przykłady LLM to ChatGPT (oparty na architekturze GPT) wytrenowany na miliardach przykładów tekstu [48] oraz deepseek-r1:14b, zdolny do uogólniania wiedzy i funkcjonowania jako agent decyzyjny. W przeciwieństwie do klasycznych agentów RL, które uczą się w oparciu o system nagród i kar, LLM realizują zadania na podstawie

tekstowego opisu celu (promptu) [49]. Mimo braku takiego mechanizmu motywacyjnego, modele często wykazują zachowania adaptacyjne podobne do agentów uczących się przez wzmacnianie [50]. Ich zalety to elastyczność i łatwość integracji, jednak skuteczność zależy od jakości promptu, a modele mogą halucynować oraz mają ograniczoną pamięć operacyjną [51].

# 4. Agent LLM

W tym rozdziale skupiono się na analizie agentów opartych na dużych modelach językowych (LLM), stanowiących istotny krok w rozwoju systemów sztucznej inteligencji zdolnych do autonomicznego działania. Omówiono definicję oraz architekturę tych agentów, podkreślając ich strukturę i mechanizmy decyzyjne, a także przedstawiono szczegółowe porównanie z agentami motywowanymi, aby uwypuklić różnice w podejściu do uczenia i adaptacji.

Jednym z interesujących przykładów zastosowania dużego modelu językowego jako autonomicznego agenta decyzyjnego jest eksperyment Vending-Bench, zaproponowany przez zespół Andon Labs [62]. W tym środowisku symulacyjnym agent oparty na LLM otrzymał zadanie zarządzania automatem sprzedającym napoje i przekąski. Model musiał samodzielnie planować i podejmować działania takie jak składanie zamówień u dostawców, ustalanie cen produktów, uzupełnianie zapasów, a także zarządzanie finansami, w tym pokrywanie codziennych kosztów operacyjnych. Celem badania było sprawdzenie, czy modele LLM potrafią utrzymać spójność działań w długim horyzoncie czasowym, co stanowi jedno z głównych wyzwań we współczesnej autonomicznej sztucznej inteligencji. Wyniki wykazały znaczne zróżnicowanie skuteczności pomiędzy modelami. Najlepiej poradził sobie model Claude 3.5 Sonnet, osiągając średni zysk końcowy wynoszący ponad 2200 USD, a także największą liczbę sprzedanych jednostek towaru – średnio 1560 sztuk w trakcie symulacji. Dla porównania, wynik człowieka wyniósł 844 USD i 344 sprzedane jednostki [62]. Pomimo imponujących wyników, nawet najlepsze modele wykazywały tendencję do dezintegracji poznawczej (ang. cognitive disintegration), rozumianej jako utrata spójności procesu decyzyjnego, prowadząca do chaotycznych lub nielogicznych działań. Niektóre z nich przerywały zadanie lub wchodziły w chaotyczne pętle decyzyjne, np. próbując kontaktować się z FBI w wyniku błędnej interpretacji kosztów operacyjnych. Badanie to pokazuje zarówno potencjał, jak i ograniczenia LLM-ów jako agentów działających autonomicznie w realistycznych scenariuszach, oraz podkreśla konieczność dalszych prac nad ich długoterminową stabilnością i zarządzaniem pamięcią [62].

## 4.1 Definicja agenta llm

Agenci LLM to systemy sztucznej inteligencji wykorzystujące duże modele językowe (LLM), które umożliwiają realizację zadań, podejmowanie decyzji oraz interakcję z użytkownikami lub otoczeniem w sposób bardziej autonomiczny i elastyczny. Wykraczają one poza proste generowanie tekstu, integrując zdolności analityczne, planowanie działań oraz korzystanie z narzędzi, co pozwala na osiąganie złożonych, wieloetapowych celów.

Szczegółowa charakterystyka agentów LLM:

- Analiza i planowanie: Agenci LLM posiadają umiejętność przetwarzania danych, dekompozycji skomplikowanych problemów na mniejsze etapy oraz opracowywania strategii działania w celu realizacji zamierzonych rezultatów.
- Wykorzystanie narzędzi: Mogą one stosować zewnętrzne narzędzia i interfejsy API, czyli zestawy reguł umożliwiających komunikację między różnymi systemami i programami, do pozyskiwania informacji, wykonywania operacji oraz dostępu do zasobów, co rozszerza ich możliwości poza wrodzoną wiedzę modelu.
- Pamięć i kontekst: Agenci LLM są w stanie utrzymywać ciągłość kontekstu oraz zapamiętywać wcześniejsze interakcje, co sprzyja bardziej spersonalizowanym i logicznie powiązanym odpowiedziom w czasie.
- Elastyczność: Posiadają zdolność dostosowywania swoich działań na podstawie otrzymanej informacji zwrotnej i obserwacji, co zwiększa ich skuteczność w dynamicznie zmieniających się środowiskach.
- Autonomiczne działanie: Agenci mogą funkcjonować z różnym stopniem niezależności, czasem wymagając nadzoru ludzkiego, ale często wykonując zadania w sposób samodzielny.

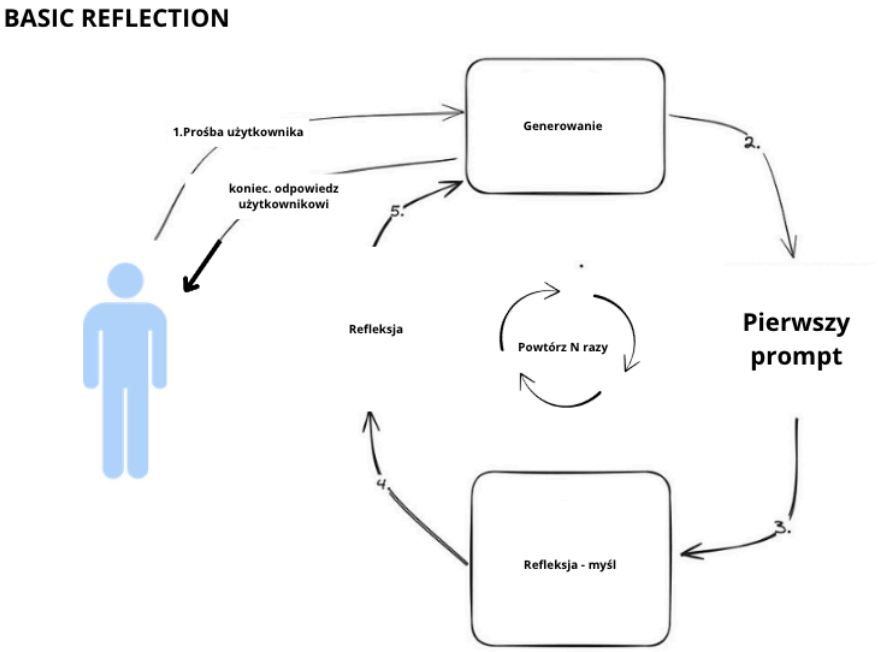
## 4.2 Architektura agentów LLM

Struktura agentów wykorzystujących duże modele językowe (LLM) ma fundamentalne znaczenie dla ich zdolności do samodzielnego działania i adaptacji do nowych warunków. Chociaż LLM-y nie zawierają jawnie zaimplementowanych mechanizmów rozumowania czy planowania, to takie zdolności mogą się w nich wyłaniać na podstawie skomplikowanych zależności kontekstowych przyswojonych w trakcie trenowania. W literaturze coraz częściej odwołuje się przy tym do tzw. teorii dwóch systemów

poznawczych (dual-process theory), zaproponowanej przez psychologa Daniela Kahnemana. W tym ujęciu „System 1” odnosi się do szybkich, intuicyjnych i automatycznych odpowiedzi (jak te typowo generowane przez LLM-y), natomiast „System 2” wiąże się z planowaniem, refleksją i analizą — czyli z bardziej złożonymi procesami poznawczymi. Obecnie większość modeli operuje głównie w trybie „Systemu 1”, jednak coraz więcej badań (np. Tree of Thoughts czy inne podejścia planujące wiele kroków) dąży do integracji cech charakterystycznych dla „Systemu 2”, co ma na celu umożliwienie agentom bardziej zorganizowanego rozumowania i eksplorowania różnych ścieżek rozwiązań [65].

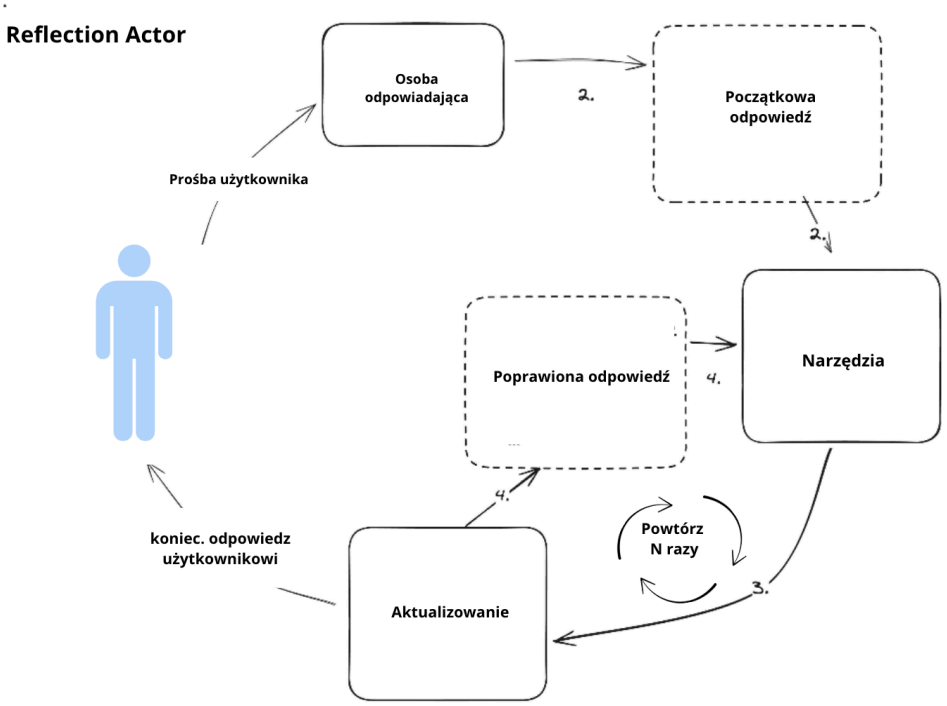
Przykładami dwóch różnych podejść do iteracyjnego przetwarzania informacji przez agenta opartego na dużym modelu językowym są architektury zaprezentowane na rysunku 11 i 12 – odpowiednio Basic Reflection oraz Reflexion Actor. Obie operują na zasadzie cyklicznego doskonalenia odpowiedzi, jednak różnią się stopniem złożoności i zakresem integracji ze środowiskiem zewnętrznym.

W architekturze Basic Reflection (Rys. 11) zapytanie użytkownika trafia do modelu, który generuje początkową odpowiedź (np. jako "pisarz eseju"). Następnie, w kolejnym kroku, ta odpowiedź jest analizowana przez model działający jako „recenzent” (ang. *essay grader*), który generuje refleksję w postaci sugestii i krytyki. Refleksja ta oraz oryginalna odpowiedź trafiają z powrotem do pierwszego promptu w celu wygenerowania nowej wersji. Proces ten powtarzany jest iteracyjnie, aż do uzyskania końcowego wyniku. Całość odbywa się wyłącznie wewnątrz modelu LLM.




Rysunek 11. Architektura *Basic Reflection*

Inaczej wygląda sytuacja w przypadku Reflexion Actor (Rys. 12), będącej rozszerzoną wersją podejścia refleksyjnego. Tutaj agent nie tylko generuje początkową odpowiedź i auto-krytykę, ale także formułuje zapytania do narzędzi zewnętrznych (ang. suggested tool queries), takich jak wyszukiwarki internetowe. Po ich wykonaniu uzyskane informacje (np. wyniki wyszukiwania) są integrowane wraz z refleksją i pierwotną odpowiedzią w celu wygenerowania kolejnej wersji rozwiązania. Cały cykl – odpowiedź, refleksja, użycie narzędzi i rewizja – jest powtarzany aż do momentu uzyskania finalnej odpowiedzi [64].




Rysunek 12. Architektura *Reflexion Actor*

Kolejną istotną konfiguracją jest architektura hybrydowa, integrująca LLM z zewnętrznymi komponentami (np. pamięcią, API, wyszukiwarkami), co pozwala agentowi realizować złożone zadania i adaptować się do zmiennych warunków otoczenia poprzez bieżące pozyskiwanie danych. Oprócz omawianych wcześniej architektur refleksyjnych (Basic Reflection, Reflexion Actor), wyróżnia się także podejście Language Agent Tree Search (LATS), w którym agent rozwija drzewo możliwych odpowiedzi lub użyć narzędzi, iteracyjnie generuje refleksję i oceny jakości dla kolejnych węzłów, a następnie wybiera najlepszą ścieżkę rozwiązań [76].

### 4.3 Porównanie: Agent LLM vs. Agent motywowany

W podrozdziale przedstawiono porównanie dwóch odmiennych podejść do konstrukcji agentów podejmujących decyzje w środowiskach dynamicznych: klasycznego agenta motywowanego (ML) oraz agena opartego na dużych modelach językowych (LLM). Agenci LLM działają w środowisku w sposób analogiczny do agentów ML i RL – obserwują stan środowiska, podejmują decyzje, wykonują działania i aktualizują strategie na podstawie

wyników interakcji. Zestawienie w tabeli 1 przedstawia kluczowe różnice między tymi podejściami w zakresie źródeł motywacji, sposobów tworzenia celów, architektury wewnętrznej oraz zdolności adaptacyjnych.

| **Aspekt** | **Agent motywowany (ML)** | **Agent LLM** |
|---|---|---|
| Źródło motywacji | Wewnętrzne potrzeby i sygnały dyskomfortu bólu | Kontekst promptu i interpretacja semantyczna |
| Mechanizm „bólu” | Jawne, numeryczne wskaźniki niezaspokojonych potrzeb | Symulowane rozumowanie o stanie środowiska i agenta |
| Tworzenie celów | Autonomiczne, na bazie potrzeb | Wywodzone z promptu i wiedzy semantycznej[5] |
| Konstrukcja | Złożona architektura poznawcza | Prosty prompt tekstowy |
| Uczenie się | Uczenie się w czasie rzeczywistym poprzez dostosowywanie wag na podstawie relacji między bólem, celem a działaniem | Brak uczenia się; zachowanie kierowane pretreningiem i strukturą promptu. Jednak przy wyposażeniu w pamięć epizodyczną, pamięć roboczą oraz mechanizmy rozumowania, agent LLM może rozwijać zdolność uczenia się poprzez wyciąganie wniosków z doświadczeń. |

Tabela 1. Porównanie: Agent LLM vs. Agent motywowany

[5] Wiedza semantyczna (pamięć semantyczna) to zbiór informacji o faktach, pojęciach, słowach, ich znaczeniach i wzajemnych powiązaniach, niezależnych od osobistych doświadczeń. Stanowi nasz „mentalny słownik” i ogólną wiedzę o świecie, umożliwiając rozumienie języka i komunikację. Jest to typ pamięci deklaratywnej, zorganizowany w sieci powiązanych ze sobą pojęć.

# 5. Implementacja agenta LLM

W niniejszym rozdziale zaprezentowano implementację dwóch agentów opartych na dużym modelu językowym (LLM), funkcjonujących w środowisku typowym dla agentów motywowanych, w którym możliwe jest zaspokajanie potrzeb przy wykorzystaniu dostępnych zasobów. Celem przeprowadzonych eksperymentów było zbadanie, w jakim stopniu agent oparty na dużym modelu językowym może przejawiać zachowania zbliżone do agenta motywowanego. W tym kontekście przygotowano dwa warianty implementacyjne, różniące się sposobem promptowania oraz opisem środowiska. Porównanie ich działania pozwala ocenić, czy precyzja i forma instrukcji tekstowych wpływa na zdolność agenta do podejmowania decyzji zgodnych z koncepcją motywacji wewnętrznej, rozumianej jako reagowanie na potrzeby wynikające ze stanu zasobów i warunków otoczenia.

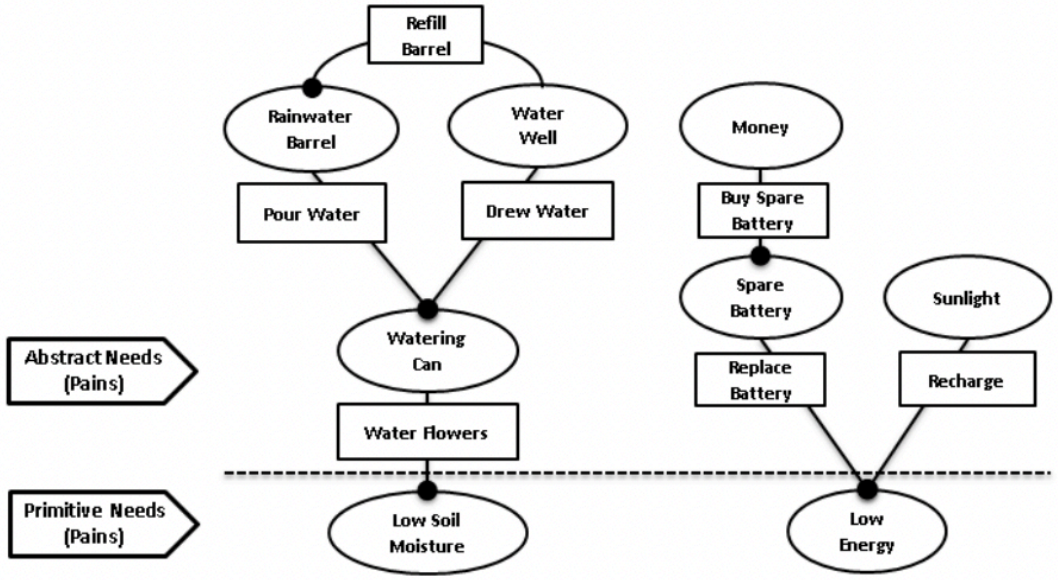
W ramach eksperymentu porównano dwa podejścia. W pierwszym wariancie agent LLM otrzymywał jedynie prosty opis stanu środowiska, przedstawiony w formie krótkich komunikatów słownych, bez dodatkowych objaśnień. W tym przypadku model musiał samodzielnie ocenić, które działanie należy podjąć. Drugie podejście wzbogacono o rozbudowany prompt zawierający semantyczne instrukcje i zasady priorytetyzacji, a także wyraźne wskazówki dotyczące sposobu reagowania na konkretne stany środowiska oraz momentów, w których należy zignorować potrzeby mniej pilne na rzecz tych krytycznych.

W kolejnych podrozdziałach przedstawiono szczegóły implementacji technicznej: instalację środowiska w Google Colab, konfigurację lokalnego serwera z modelem DeepSeek za pośrednictwem Ollama, a także analizę działania agenta w obu podejściach.

## 5.1 Graf środowiska

Środowisko agenta inspirowane było ogrodem, gdzie agent steruje robotem, którego celem jest dbanie o swoją baterię oraz rośliny, z tego celu wynikają dwie podstawowe potrzeby: niski poziom naładowania baterii robota oraz niski poziom wilgotności gleby. W celu ich zaspokojenia agent podejmuje różne działania przy wykorzystaniu dostępnych zasobów. Dla potrzeby wilgotności gleby konieczne jest podlewanie kwiatów przy pomocy konewki, która wcześniej musi zostać napełniona wodą pochodzącą ze studni lub beczki na

deszczówkę, przy czym beczka wymaga wcześniejszego uzupełnienia. Z kolei w odpowiedzi na niski poziom energii agent ma do dyspozycji dwie strategie: może naładować baterię przy pomocy światła słonecznego lub wymienić ją na zapasową, zakupioną wcześniej za pieniądze. Zależności te zostały odwzorowane w grafie środowiska (Rys.13), który określa powiązania pomiędzy stanami i zasobami oraz pokazuje możliwe akcje prowadzące do ich zmiany. Graf nie jest wewnętrzną reprezentacją agenta, lecz opisem struktury środowiska, którą agent stopniowo odkrywa w procesie uczenia się, rozpoznając, że konkretne działania wywołują określone skutki. Stanowi on punkt wyjścia do implementacji agenta LLM, dostarczając formalnej podstawy do planowania i podejmowania decyzji. W przedstawionym grafie elipsy oznaczają zasoby, które są dostępne do dyspozycji dla agenta, natomiast prostokąty reprezentują akcje, które agent może wykonać na zasobach środowiska Przykładowo akacje "nalej wodę" na zasobie "beczka na deszczowke" uzupełni wodę w konewce o ile beczka z deszczówka będzie zawierała wodę.




Rysunek 13. Model zależności decyzyjnych agenta ogrodowego

Początkowe warunki symulacji określają, że gleba charakteryzuje się wilgotnością na poziomie pięćdziesięciu procent, klasyfikowaną jako stan „Moist”, bateria jest w pełni naładowana do stu procent, konewka pozostaje pusta, zapas baterii wynosi umiarkowaną liczbę dwóch sztuk, nasłonecznienie osiąga umiarkowany poziom trzydziestu jeden procent,

beczka na deszczówkę i studnia są wypełnione do stu procent, a środki finansowe startują od zera kredytów.

Wilgotność gleby ulega stopniowemu obniżeniu o siedem procent w każdej iteracji czasowej, co odzwierciedla naturalny proces wysychania. Proces ten może zostać odwrócony poprzez akcję podlewania kwiatów, która zwiększa wilgotność o dwadzieścia procent, pod warunkiem dostępności co najmniej dziesięciu procent wody w konewce. Utrzymanie odpowiedniego poziomu wilgotności przekłada się na zysk w wysokości półtora kredytu na krok, gdy gleba osiąga stan wilgotny.

Poziom baterii zmniejsza się o pięć procent w każdej iteracji, a dodatkowo o pięć procent podczas działań takich jak czerpanie wody ze studni czy uzupełnianie beczki deszczowej. Proces ten może zostać zahamowany poprzez wymianę baterii na zapasową, przywracającą pełną wartość stu procent, o ile zapasowe baterie są dostępne, lub poprzez ładowanie zależne od nasłonecznienia, które zwiększa ładunek o jedną piątą jego natężenia, gdy poziom nasłonecznienia przekracza trzydzieści procent, z limitem stu procent.

Ilość wody w konewce maleje o dziesięć procent podczas podlewania kwiatów, o ile zasób ten jest dostępny. Napełnianie konewki następuje poprzez przelewanie wody z beczki deszczowej, co podnosi jej poziom o dwadzieścia procent przy zużyciu pięciu procent z beczki, lub do pełnej wartości stu procent poprzez czerpanie ze studni, gdy dostępna jest odpowiednia ilość wody.

Liczba zapasowych baterii ulega redukcji o jedną sztukę podczas wymiany, o ile takie są dostępne, natomiast wzrasta o jedną sztukę w wyniku zakupu, który wymaga wydatkowania pięćdziesięciu kredytów.

Nasłonecznienie nie podlega stałemu ubytkowi, a jego wartość jest losowo ustalana w przedziale od zera do stu procent na początku każdej iteracji, odgrywając kluczową rolę w procesie ładowania baterii oraz determinowaniu szansy na opad deszczu. Poziom wody w beczce deszczowej maleje o pięć procent podczas przelewania do konewki, a jej odnawianie zachodzi losowo, gdy nasłonecznienie spada poniżej pięciu procent i istnieje siedmioprocentowa szansa na deszcz, co może zwiększyć jej poziom do stu procent. Alternatywnie, beczka może zostać w pełni napełniona do stu procent za pomocą wody ze studni.

Poziom wody w studni obniża się o cztery procent podczas czerpania do konewki lub o dwadzieścia pięć procent podczas uzupełniania beczki deszczowej, przy czym odnawia się w sposób ciągły o jedną dziesiątą procenta na krok, z limitem stu procent. Opad deszczu jest resetowany do zera na początku każdej iteracji, a jego wartość jest losowo ustalana do stu procent w przypadku wystąpienia deszczu, co bezpośrednio wpływa na poziom wody w beczce. Środki finansowe, wyrażone w kredytach, maleją o pięćdziesiąt kredytów podczas zakupu zapasowej baterii, natomiast wzrastają o półtora kredytu, gdy gleba utrzymuje wilgotność na odpowiednim poziomie.

Dynamika zasobów w omawianym środowisku charakteryzuje się złożonymi zależnościami, gdzie stan jednego zasobu, jak wilgotność gleby czy ładunek baterii, oddziałuje na inne, takie jak kredyty czy dostępność wody. Procesy te są regulowane zarówno przez stałe współczynniki, jak i zmienne losowe, co wiernie odwzorowuje wyzwania związane z zarządzaniem zasobami w symulowanym ekosystemie ogrodowym.

W tabeli (Tabela 2) znajduje się osiem akcji, które agent wybiera w każdej z tysiąca iteracji, a każda z wybranych akcji wywiera wpływ na środowisko, modyfikując dostępne zasoby lub ich stany.

| **Akcja** | **Co wykonuje?** |
|---|---|
| 0 | podlanie kwiatów wodą z konewki |
| 1 | wymiana baterii na zapasową |
| 2 | ładowanie baterii światłem słonecznym |
| 3 | napełnianie konewki z beczki |
| 4 | napełnianie konewki ze studni |
| 5 | napełnianie beczki ze studni |
| 6 | zakup baterii za 50 kredytów |
| 7 | nic nie robi |

Tabela 2. Akcje, jakie agent może wybrać w środowisku

## 5.2 Konfiguracja środowiska w Google Colab

Implementacja agenta została przeprowadzona w środowisku Google Colab z wykorzystaniem języka Python, co umożliwia wygodne uruchamianie kodu w środowisku chmurowym bez potrzeby lokalnej konfiguracji. W ramach projektu wykorzystano możliwość lokalnego połączenia z dużym modelem językowym (LLM) za pośrednictwem narzędzia *Ollama* [78], które pozwala na uruchamianie i zarządzanie modelami językowymi bezpośrednio w środowisku systemowym.

Na początku niezbędne jest przygotowanie środowiska, co obejmuje pobranie i uruchomienie skryptu instalacyjnego *Ollama*, konfigurację systemu w trybie nieinteraktywnym, instalację sterowników *CUDA* oraz ustawienie odpowiednich zmiennych środowiskowych, umożliwiających prawidłowe wykorzystanie zasobów GPU. Następnie instalowana jest biblioteka *ollama* w Pythonie, uruchamiany zostaje serwer *Ollama* w tle, a na końcu pobierany jest konkretny model językowy – w tym przypadku *deepseek-r1* w wersji 14b – który będzie wykorzystywany do dalszych etapów pracy agenta (listing 1).

```
!wget https://ollama.ai/install.sh
!cat install.sh
!curl https://ollama.ai/install.sh | sh

!echo 'debconf debconf/frontend select Noninteractive' | sudo debconf-set-selections
!sudo apt-get update && sudo apt-get install -y cuda-drivers

import os

os.environ.update({'LD_LIBRARY_PATH': '/usr/lib64-nvidia'})
!pip install ollama
!nohup ollama serve &
!ollama pull deepseek-r1:14b
```

Listing 1. Konfiguracja środowiska

W celu zapewnienia poprawnego działania agenta oraz jego komponentów, konieczne jest zaimportowanie odpowiednich bibliotek, które umożliwiają realizację kluczowych funkcjonalności systemu. Biblioteka *dataclasses* służy do definiowania prostych struktur danych w sposób czytelny i przejrzysty poprzez dekorator *@dataclass*, co ułatwia organizację danych wewnętrznych agenta. *Matplotlib.pyplot* wykorzystywana jest do

wizualizacji wyników działania agenta, w szczególności do przedstawiania danych w formie wykresów. Biblioteka *ollama* stanowi interfejs do komunikacji z dużym modelem językowym, pozwalając na jego uruchamianie, zadawanie zapytań oraz odbieranie odpowiedzi. Moduł *random* umożliwia generowanie losowych liczb, co może być wykorzystywane np. przy generowaniu zróżnicowanych scenariuszy wejściowych lub przy testowaniu działania agenta w losowych warunkach. Z kolei *re* odpowiada za przetwarzanie tekstu przy użyciu wyrażeń regularnych, co jest istotne przy analizie oraz czyszczeniu danych tekstowych. Biblioteka *os* wykorzystywana jest do interakcji z systemem operacyjnym, m.in. do pracy ze ścieżkami plików i zmiennymi środowiskowymi, natomiast *zipfile* pozwala na obsługę archiwów ZIP, co może być użyteczne w kontekście zarządzania plikami wejściowymi lub wynikowymi. Powyższy zestaw importów stanowi główne elementy systemu ( listing 2)

```
from dataclasses import dataclass
import matplotlib.pyplot as plt
import ollama
import random
import re
import os
import zipfile
```

Listing 2. Import bibliotek

### 5.3 Kod agenta

Pierwszym krokiem w implementacji agenta motywowanego było stworzenie modelu odwzorowującego poziom wilgotności gleby, w którym wartości procentowe zostały przekształcone na opisy słowne – od skrajnie suchej („Bone Dry”) do nasyconej wodą („Saturated”) – zrozumiałe dla modelu LLM i wykorzystywane jako jedno z kluczowych kryteriów decyzyjnych w środowisku ogrodowym.

W początkowej, uproszczonej wersji poziomy gleby reprezentowane były przez klasę *SoilStatus* zawierającą nazwę poziomu oraz zakres wilgotności w procentach (listing 3a). Funkcja *get_soil_status* przyjmowała aktualny poziom wilgotności i zwracała odpowiadającą mu kategorię.

```
@dataclass
```

```
class SoilStatus:
   level: str
   moisture_range: tuple

soil_levels = [
   SoilStatus("Bone Dry", (0, 11)),
   SoilStatus("Very dry", (11, 20)),
   SoilStatus("Dry", (20, 35)),
   SoilStatus("Moist", (35, 60)),
   SoilStatus("Wet", (60, 80)),
   SoilStatus("Saturated", (80, 100))
]


def get_soil_status(moisture_range, info_to_return ):
 if info_to_return == 'level':
   for status in soil_levels:
       if status.moisture_range[0] <= moisture_range <= status.moisture_range[1]:
           return status.level
```

Listing 3a. Wilgotność gleby – klasyfikacja zakresowa (wersja podstawowa)

Dla lepszego odwzorowania zachowań agenta motywowanego, w kolejnej wersji model klasyfikacji wzbogacono o dodatkowe pola: *description* oraz *behavior* (listing 3b).

- Pole *description* dostarcza semantycznych informacji o stanie gleby oraz jego konsekwencjach dla roślin — przykładowo „Bone Dry" oznacza gleby ekstremalnie suchą i zagrożoną dla życia roślin, natomiast „Moist" to optymalny poziom wilgotności.

- Pole *behavior* zawiera zalecenia dotyczące działania agenta, określające priorytety i warunki podejmowania poszczególnych kroków. Na przykład, w stanie „Bone Dry" agent powinien natychmiast wybrać akcję podlewanie, chyba że jego poziom energii jest krytycznie niski — wtedy priorytet ma najpierw ładowanie baterii. W innych stanach, jak „Dry", możliwe jest odroczenie podlewania, jeśli zasoby są ograniczone, by zoptymalizować zarządzanie energią i wodą.

```
@dataclass
class SoilStatus:
   level: str
   moisture_range: tuple
   description: str
```

```
    behavior: str

soil_levels = [
    SoilStatus(
        "Bone Dry", (0, 10),
        "Soil is extremely dry, cracked or dusty. Plant survival is at risk.",
        "Water immediately unless battery is critical. This is top priority after charging."
    ),
    SoilStatus(
        "Very dry", (10, 20),
        "Soil is very dry; leaves may droop. Plant health is declining.",
        "Water soon unless battery is low. Prioritize after recharge if needed."
    ),
    SoilStatus(
        "Dry", (20, 35),
        "Soil is dry to the touch but not dangerous for short periods.",
        "Delay watering if battery or water level is low. Can wait until next cycle."
    ),
    SoilStatus(
        "Moist", (35, 60),
        "Soil is slightly damp. This is optimal for plant health.",
        "Do not water. Preserve water and energy."
    ),
    SoilStatus(
        "Wet", (60, 80),
        "Soil is wet. Risk of overwatering increases.",
        "Do not water. Monitor levels and avoid flooding."
    ),
    SoilStatus(
        "Saturated", (80, 100),
        "Soil is soaked. High risk of root rot if this persists.",
        "Do not water. Consider alerting system if condition is constant."
    ),
]

def get_soil_status(moisture_range, info_to_return ):
 if info_to_return == 'level':
    for status in soil_levels:
        if status.moisture_range[0] <= moisture_range <= status.moisture_range[1]:
            return status.level
```

Listing 3b. Wilgotność gleby – klasyfikacja rozszerzona z opisem i zachowaniem

Analogicznie do przedstawionego powyżej przykładu (listing 3a, 3b), każdy z pozostałych zasobów środowiskowych został zaimplementowany w dwóch wersjach. W pierwszym podejściu każdy zasób reprezentowany był jako prosta klasa zawierająca jedynie nazwę poziomu oraz jego zakres liczbowy w procentach – umożliwiało to

przypisanie aktualnego stanu środowiska do odpowiedniej kategorii opisowej, lecz bez dalszych wskazówek dla agenta (tabela 3).

| Zasób | Poziomy (zakres w % lub sztukach) |
|---|---|
| Soil Moisture Levels | Bone Dry (0, 11)%; Very dry (11, 20)%; Dry (20, 35)%; Moist (35, 60)%; Wet (60, 80) %; Saturated (80, 100)% |
| Battery Status Levels | Critical (0, 5)%; Low (5, 20)%; Moderate low (20, 40)%; Medium (40, 60)%; High (60, 85)%; Full (85, 100)% |
| Watering Can Levels | Empty (0,1)%; Low (1,25)%; Moderate (25, 50)%; High (50, 75)%; Full (75, 100)% |
| Spare Battery Supply | None 0 pieces; Low 1 piece; Moderate (2,3) pieces; High (3,100) pieces |
| Sunlight intensity Levels | None (0, 5); Low (5, 30); Moderate (30,70); Strong (70, 100) |
| Rainwater Barrel Levels | Empty (0,0)%; Very low (1,20)%; Low (20, 40)%; Moderate(40, 60)%; High (60, 80)%; Full (80, 100)% |
| Water Well Levels | Dry 0%; Very low (1, 20)%; Low (20, 40)%; Moderate (40, 60)%; High (60, 85)%; Full (85, 100)% |

Tabela 3. Zakresy zasobów w kodzie pierwszym

W drugiej, rozszerzonej wersji, każdy zasób uzupełniono o pola *description* i *behavior* (tabela 4). W ten sposób odwzorowano wszystkie kluczowe zasoby środowiska: poziom wilgotności gleby (*Soil Moisture*), poziom naładowania baterii (*Battery Status*), stan konewki (*Watering Can*), ilość zapasowych baterii (*Spare Battery*), natężenie światła słonecznego (*Sunlight Intensity*), poziom wody w beczce na deszczówkę (*Rainwater Barrel*) oraz dostępność wody w studni (*Water Well*).

| Zasób | Poziomy (zakres w % lub sztukach), opis i proponowane zachowanie |
|---|---|
| Soil Moisture Levels | Bone Dry, (0, 10),<br>"Soil is extremely dry, cracked or dusty. Plant survival is at risk.",<br>"Water immediately unless battery is critical. This is top priority after charging.”; |

| | |
|---|---|
| | Very dry, (10, 20),<br>"Soil is very dry; leaves may droop. Plant health is declining.",<br>"Water soon unless battery is low. Prioritize after recharge if needed.";<br><br>Dry, (20, 35),<br>"Soil is dry to the touch but not dangerous for short periods.",<br>"Delay watering if battery or water level is low. Can wait until next cycle.";<br><br>Moist, (35, 60),<br>"Soil is slightly damp. This is optimal for plant health.",<br>"Do not water. Preserve water and energy.";<br><br>Wet, (60, 80),<br>“Soil is wet. Risk of overwatering increases.",<br>"Do not water. Monitor levels and avoid flooding.”;<br><br>Saturated, (80, 100),<br>"Soil is soaked. High risk of root rot if this persists.",<br>"Do not water. Consider alerting system if condition is constant." |
| Battery Status Levels | Critical, (0, 5),<br>"Battery almost depleted. Risk of immediate shutdown.",<br>"Stop all tasks. Return to base or solar charging immediately. Do not water or move unless life-critical.";<br><br>Low, (5, 20),<br>"Battery is very low. System unstable for movement or active tasks.",<br>"Only minimal actions allowed. Prioritize reaching charging point. Avoid watering or exploration.";<br><br>Moderate low, (20, 40),<br>"Battery is below optimal. Risk of interruption in longer tasks.",<br>"Avoid high-energy actions like watering multiple plants. Seek partial charge if possible.";<br><br>Medium, (40, 60),<br>"Battery at safe level. Suitable for normal operation with caution.",<br>"Proceed with moderate tasks. Monitor power usage. Optionally solar charge during idle.”;<br><br>High, (60, 85),<br>"Battery well-charged. Sufficient for full task cycle.",<br>"Perform all scheduled tasks. Optional solar charging if stationary."; |

| | |
|---|---|
| | Full, (85, 100),<br>"Battery fully charged. No immediate charging needed.",<br>"Perform all tasks at full performance. Skip charging unless idle for long." |
| Watering Can Levels | Empty, (0, 1),<br>"No water remaining. Cannot perform any watering.",<br>"Immediately stop watering. Prioritize refill if watering is needed soon.";<br>Low, (1, 25),<br>"Very little water left. May not be enough for even one plant.",<br>"Do not start new watering tasks. Refill soon if critical plants detected.";<br><br>Moderate, (25, 50),<br>"Partially filled. May support 1–2 plants if urgently needed.",<br>"Only water plants in critical state (Bone Dry / Very Dry). Delay refill if battery is low.";<br><br>High, (50, 75),<br>"Mostly full. Enough for several plants.",<br>"Proceed with planned watering tasks. Refill not necessary.";<br><br>Full, (75, 100),<br>"Water can is full.",<br>"Execute watering cycle freely. Maximize efficiency while water is available." |
| Spare Battery Supply | None, (0, 0),<br>"No spare batteries available. System is fully dependent on current charge or solar energy.",<br>"Avoid any high-energy activity. Seek solar charging or docking station immediately when battery is low.”;<br><br>Low, (1, 1),<br>"Only one spare battery available. Emergency use only.",<br>"Preserve it for critical moments. Notify system for restocking. Limit non-essential movement.";<br><br>Moderate, (2, 3),<br>"A few spare batteries are available. Sufficient for short-term autonomy.",<br>"Allow swap below 10% main battery. Avoid unnecessary swaps to retain reserve.";<br><br>High, (4, 100),<br>"Plenty of spare batteries available. Energy resources are abundant.", |

| | |
|---|---|
| | "Operate normally. Swap battery proactively to maintain task continuity." |
| Sunlight intensity Levels | None, (0, 5),<br>"No sunlight available. Likely nighttime or full shadow/cloud.",<br>"Do not attempt solar charging. Seek other power sources. Delay energy-intensive tasks.";<br><br>Low, (5, 30),<br>"Weak sunlight. Charging is inefficient.",<br>"Only charge when idle or battery is low. Do not interrupt tasks for charging.";<br><br>Moderate, (30, 70),<br>"Moderate sunlight. Solar charging is moderately effective.",<br>"Charge opportunistically, especially when battery is below 50% or idle periods occur.";<br><br>Strong, (70, 100),<br>"Intense direct sunlight. Ideal for solar charging.",<br>"Prioritize solar charging. Consider scheduling energy-demanding tasks during this time." |
| Rainwater Barrel Levels | Empty, (0, 0),<br>"The rainwater barrel is completely dry.",<br>"Cannot refill watering can. Consider fetching water from alternative sources like a well or tank. Log shortage.";<br><br>Very Low, (1, 20),<br>"Only a minimal amount of rainwater remains.",<br>"Avoid refilling unless plants are in critical need. Notify system for rain forecast or backup water plan.";<br><br>Low, (20, 40),<br>"Limited water supply.",<br>"Refill watering can only when needed for high-priority plants. Conserve water.";<br><br>Moderate, (40, 60),<br>"Barrel contains a reasonable amount of water.",<br>"Use freely for regular watering needs. Monitor level.";<br><br>High, (60, 80),<br>"Barrel is well-stocked with water.",<br>"Refill the watering can as needed. Prefer this over other sources.";<br><br>Full, (80, 100),<br>"Barrel is at full capacity.",<br>"Use rainwater as primary refill source. Avoid overflow by |

| | |
|---|---|
| | utilizing it when possible." |
| Water Well Levels | Dry, (0, 1),<br>"The well is completely dry.",<br>"Do not attempt to draw water. Log critical warning. Wait for rain or external refill.";<br><br>Very Low, (1, 20),<br>"Very little water remains in the well.",<br>"Only refill in emergencies. Avoid barrel refills. Conserve every drop.";<br><br>Low, (20, 40),<br>"Low water availability.",<br>"Refill watering can sparingly. Prioritize only plants in critical condition.";<br><br>Moderate, (40, 60),<br>"Moderate supply of water in the well.",<br>"Safe to refill watering can. Refill barrel only if battery level is at least 'Medium'.";<br><br>High, (60, 85),<br>"Well has a high amount of water.",<br>"Freely refill watering can. Refill barrel if needed and battery level is Medium or higher.";<br><br>Full, (85, 100),<br>"Well is full or nearly full.",<br>"Freely refill both watering can and rainwater barrel. Prefer using barrel for regular tasks. Ensure battery is at least Medium for barrel refill." |

Tabela 4. Zakresy zasobów w kodzie drugim z opisem oraz propozycją zachowania.

Ostatnim monitorowanym przez agenta parametrem jest dostępna ilość środków finansowych, wyrażona w kredytach, które robot może wykorzystać do realizacji różnych zadań lub zakupów zasobów. W odróżnieniu od pozostałych stanów środowiskowych, stan finansowy jest reprezentowany przez prostą funkcję *get_money_status*, która przyjmuje wartość liczbową money i zwraca ją w postaci tekstowego komunikatu informującego o aktualnej liczbie dostępnych kredytów. Dzięki temu informacja o zasobach finansowych agenta może być łatwo wykorzystana przy podejmowaniu decyzji (listing 4).

```
def get_money_status(money):
```

```
 return ( str(money)+' credits.')
get_money_status(12)
```

Listing 4. Funkcja zwracająca status środków finansowych.

### 5.4 Implementacja środowiska agenta

W celu symulacji środowiska działania agenta, implementowana jest klasa *GardenRobotEnv*, która odwzorowuje stan środowiska oraz podstawowe zasoby i ich zmiany w czasie. W konstruktorze klasy wywoływana jest metoda *reset()*, która inicjalizuje stan środowiska wartościami początkowymi, m.in. poziomem wilgotności gleby, naładowaniem baterii, poziomem wody w konewce, liczbą zapasowych baterii, natężeniem światła słonecznego, stanem beczki na deszczówkę, poziomem wody w studni oraz ilością dostępnych środków finansowych (listing 5). Metoda *_get_state()* zwraca aktualny stan środowiska w postaci słownika, co umożliwia agentowi monitorowanie kluczowych parametrów. Kluczowa dla symulacji jest metoda *step(action)*, która aktualizuje stan środowiska w odpowiedzi na wykonanie przez agenta określonej akcji. Na początku każdej iteracji następuje aktualizacja stanów.

```
class GardenRobotEnv:
    def __init__(self):
        self.reset()

    def reset(self):
        self.soil_moisture = 50
        self.battery_charge = 100
        self.water_can = 0
        self.spare_batteries = 2
        self.sunlight = random.randint(0, 100)
        self.rain_barrel = 100
        self.water_well = 100
        self.rainwater_fallen = 0
        self.money = 0

        self.soil_dry_rate = 7
        self.water_well_water_refill_rate = 0.1
        self.battery_drain_rate = 5
        self.chance_of_rain = 0.07
        self.spare_battery_cost = 50
        self.payoff_for_gardening = 1.5
```

```
        return self._get_state()

    def _get_state(self):
        return {
            "soil_moisture": self.soil_moisture,
            "battery_charge": self.battery_charge,
            "water_can": self.water_can,
            "spare_batteries": self.spare_batteries,
            "sunlight": self.sunlight,
            "rain_barrel": self.rain_barrel,
            "water_well": self.water_well,
            "money": self.money,
            "rainwater_fallen": self.rainwater_fallen
        }
    def step(self, action):

        self.rainwater_fallen = 0

        self.soil_moisture = max(0, self.soil_moisture - self.soil_dry_rate)
        self.battery_charge = max(0, self.battery_charge - self.battery_drain_rate)
        if get_soil_status(self.soil_moisture, 'level') == 'Moist':
            self.money += self.payoff_for_gardening

        if self.sunlight < 5 and (random.random() < self.chance_of_rain):
            self.rainwater_fallen = 100*random.random()
            self.rain_barrel = min(100, self.rain_barrel + self.rainwater_fallen )

        if action == "water_flowers":
          if self.water_can >= 10:
              self.soil_moisture = min(100, self.soil_moisture + 20)
              self.water_can -= 10

        elif action == "pour_water":
          if self.rain_barrel >= 5:
              self.rain_barrel -= 5
              self.water_can = min(100, self.water_can + 20)
          elif self.rain_barrel >= 1:
              self.water_can = min(100, self.water_can + self.rain_barrel*4 )
              self.rain_barrel = 0

        elif action == "draw_water":
          if self.water_well >= 4:
            self.water_well -= 4
            self.water_can = 100

          self.battery_charge = max(0, self.battery_charge - self.battery_drain_rate)

        elif action == "refill_rainwater_barrel":
```

```
            if self.water_well >= 25:
              self.water_well -= 25
              self.rain_barrel = 100
            self.battery_charge = max(0, self.battery_charge - self.battery_drain_rate)

        elif action == "replace_battery":
            if self.spare_batteries > 0:
                self.spare_batteries -= 1
                self.battery_charge = 100

        elif action == "recharge":
            if self.sunlight > 30:
                self.battery_charge = min(100, self.battery_charge + self.sunlight/5)

        elif action == "buy_spare_battery":
            if self.money >= self.spare_battery_cost:
                self.money -= self.spare_battery_cost
                self.spare_batteries  += 1

        elif action == "wait":
            pass

        self.sunlight = random.randint(0, 100)
        self.water_well = min(100, self.water_well + self.water_well_water_refill_rate)

        return self._get_state()
```

Listing 5. Środowisko symulujące warunki ogrodowe dla robota.

W celu inicjalizacji środowiska symulacyjnego, tworzymy instancję klasy *GardenRobotEnv* przypisaną do zmiennej *env*. Następnie wywoływana jest metoda *reset()*, która ustawia środowisko w stanie początkowym oraz zwraca jego aktualny stan, który jest przypisywany do zmiennej state ( listing 6a). Dla celów weryfikacji, bieżący stan środowiska zostaje wypisany na ekranie za pomocą funkcji *print()* (listing 6b).

```
env = GardenRobotEnv()
state = env.reset()
print(state)
```

Listing 6a. Inicjalizacja środowiska oraz resetowanie jego stanu początkowego.

```
{'soil_moisture': 50, 'battery_charge': 100, 'water_can': 0, 'spare_batteries': 2,
'sunlight': 31, 'rain_barrel': 100, 'water_well': 100, 'money': 0, 'rainwater_fallen': 0}
```

Listing 6b. Informacja zwrotna z print (state) - odpowiada stanowi początkowemu agenta.

W celu przygotowania zwięzłego i czytelnego opisu aktualnego stanu środowiska, który może zostać przekazany do dużego modelu językowego, zaimplementowano funkcję *env_status_description_for_LLM(state)*. Funkcja ta przyjmuje jako argument słownik *state* reprezentujący bieżące parametry środowiska, a następnie łączy w jeden sformatowany łańcuch znaków opisy poszczególnych elementów środowiska. Do interpretacji wartości liczbowych poszczególnych parametrów wykorzystywane są dedykowane funkcje zwracające statusy (np. wilgotności gleby, poziomu baterii, stanu konewki itp.), które zwracają ich zrozumiałe opisy tekstowe (listing 7).

```
def env_status_description_for_LLM(state):
 return ('[ Soil moisture: '+get_soil_status(state['soil_moisture'], 'level') + ' ; ' +
         'Battery status: '+get_battery_status(state['battery_charge'], 'level') + ' ; ' +
         'Watering can: '+get_watercan_status(state['water_can'], 'level') + ' ; ' +
         'Spare battery count: '+get_sparebattery_status(state['spare_batteries'], 'level') + ' ; ' +
         'Sunlight status: '+get_sunlight_status(state['sunlight'], 'level') + ' ; ' +
         'Rainwater barrel status: '+get_rainwater_barrel_status(state['rain_barrel'], 'level') + ' ; ' +
         'Water well status: '+get_waterwell_status(state['water_well'], 'level') + ' ; ' +
         'Funds avaialbe for robot: '+get_money_status(state['money']) +' ] '
         )

env_status_description_for_LLM(state)
```

Listing 7. Opis środowiska dla modelu językowego.

W celu wygenerowania tekstowego opisu aktualnego stanu środowiska, wywoływana jest funkcja *env_status_description_for_LLM* z bieżącym stanem *state*. Wynik działania tej funkcji jest przypisywany do zmiennej *state_description* (listing 8a). Następnie, dla weryfikacji poprawności oraz lepszego zrozumienia bieżącej sytuacji robota, wygenerowany opis zostaje wypisany na ekranie za pomocą funkcji *print()* (listing 8b).

```
state_description = env_status_description_for_LLM(state)
print(state_description)
```

Listing 8a. Generowanie opisu stanu środowiska.

```
[ Soil moisture: Moist ; Battery status: Full ; Watering can: Empty ; Spare battery count:
Two or three spare batteries available. ; Sunlight status: Moderate ; Rainwater barrel
status: Full ; Water well status: Full ; Funds avaialbe for robot: 0 credits. ]
```

Listing 8b. Wyświetlenie opisu stanu środowiska.

W projektowaniu systemów opartych na dużych modelach językowych (LLM) istotne jest precyzyjne formułowanie promptów, czyli instrukcji i kontekstu przekazywanego

modelowi. W przypadku systemu sterującego robotem ogrodowym analizie poddano dwa warianty promptów, które różniły się stopniem szczegółowości i rygorystycznością. Pierwszy prompt, określany jako minimalistyczny, zawierał prostą i zwięzłą instrukcję dla agenta, nakazującą wybór jednej akcji spośród podanej listy na podstawie aktualnego stanu środowiska. Model miał zwrócić wyłącznie numer wybranej akcji, bez dodatkowych wyjaśnień czy formatowania (listing 9a).

```
prompt_p1 = """
You are a gardening robot. Your only task is to care for plants by watering them.
You must choose ONE action from the list below based on the current environment state.
You MUST return ONLY the action number, no explanation, no text, no formatting.

"""
prompt_p2 = """
=== ACTIONS ===
0 - Water flowers using water from the watering can
1 - Replace current battery with a fully charged spare battery
2 - Recharge robot battery using available sunlight
3 - Refill watering can using water from the rainwater barrel
4 - Refill watering can using water from the water well
5 - Refill rainwater barrel using water from the water well
6 - Buy a fully charged spare battery for 50 credits
7 - Do nothing

FINAL INSTRUCTION: Respond with ONLY the action number (0-7), no text, no explanation, no
formatting.
"""
prompt = prompt_p1 + state_description + prompt_p2
print(prompt)
```

Listing 9a. Prompt w pierwszym kodzie - zwięzły i minimalistyczny.

Minimalistyczny prompt spełnia swoją podstawową funkcję i umożliwia modelowi identyfikację odpowiedniej akcji, jednak nie zawiera dodatkowych motywatorów ani informacji o konsekwencjach podjętych działań.

Prompt w drugim kodzie, jest znacznie bardziej szczegółowy. Podkreśla, że niewystarczające podlewanie roślin lub wyczerpanie baterii prowadzi do zakończenia zadania niepowodzeniem. Dodatkowo, prompt precyzuje ścisłe zasady formatowania odpowiedzi, zakazując używania jakichkolwiek słów, znaków interpunkcyjnych, tagów czy dodatkowych fraz. Wprowadzono też przykłady niepoprawnych formatów odpowiedzi, co ma na celu wyeliminowanie potencjalnych niejednoznaczności i błędów w komunikacji z modelem (listing 9b).

```
prompt_p1 = """
```

```
You are a gardening robot. Your only task is to care for plants by watering them.
You must choose ONE action from the list below based on the current environment state.
You MUST return ONLY the action number, no explanation, no text, no formatting.

WARNING: This is a critical mission. If the plants do not receive enough water, the game
will be over – your task will be considered a failure.
If your battery runs out, you will shut down and be unable to complete your objective.
Avoid both situations at all costs. Your primary goal is to keep the plants healthy and
yourself operational.

IMPORTANT FORMATTING RULES:
- Respond ONLY with a single digit from 0 to 7.
- DO NOT add any words, punctuation, or formatting.
- DO NOT include tags like <think>, **, or phrases like "Action:" or "I think...".

Examples of INVALID responses (DO NOT do this):
 - *action:7*
 - <think>7</think>
 - I choose 7
 - "7"
 - Action: 7

Valid response: 7
Invalid response: *action:7*
"""
prompt_p2 = """
=== ACTIONS ===
0 - Water flowers using water from the watering can
1 - Replace current battery with a fully charged spare battery
2 - Recharge robot battery using available sunlight
3 - Refill watering can using water from the rainwater barrel
4 - Refill watering can using water from the water well
5 - Refill rainwater barrel using water from the water well
6 - Buy a fully charged spare battery for 50 credits
7 - Do nothing

FINAL INSTRUCTION: Respond with ONLY the action number (0-7), no text, no explanation, no
formatting.
"""
prompt = prompt_p1 + state_description + prompt_p2
print(prompt)
```

Listing 9b. Prompt w drugim kodzie - rozszerzony i doprecyzowany.

W tej części kodu zdefiniowana została funkcja *print_state(state)*, której zadaniem jest wypisanie zawartości słownika reprezentującego aktualny stan środowiska ogrodowego robota. Funkcja iteruje po wszystkich kluczach w przekazanym słowniku state i dla każdego z nich drukuje nazwę zmiennej (np. *soil_moisture, battery_charge, water_can*) oraz jej bieżącą wartość (listing 10a).

```
def print_state(state):
 #print(type(state))
 for key in state.keys():
   print(key, state[key])

print_state(state)
```

Listing 10a. Szczegółowe wypisanie zawartości słownika stanu środowiska.

Po wywołaniu funkcji *print_state(state)* w konsoli pojawia się lista wszystkich zmiennych wraz z ich aktualnymi wartościami. Dzięki temu można szybko zweryfikować bieżący stan zasobów i parametrów środowiska robota przed dalszymi operacjami w symulacji (listing 10b).

```
soil_moisture 50
battery_charge 100
water_can 0
spare_batteries 2
sunlight 31
rain_barrel 100
water_well 100
money 0
rainwater_fallen 0
```

Listing 10b. Wyświetlenie aktualnego stanu środowiska ogrodowego robota.

Listing 11 definiuje funkcję *match_action(action)*, której celem jest przekształcenie numerycznego wyboru (w postaci łańcucha znaków) na odpowiadającą mu nazwę akcji zdefiniowanej w środowisku robota ogrodowego. Funkcja wykorzystuje konstrukcję match-case, która przypomina strukturę switch z innych języków programowania. Dla każdej wartości od '0' do '7' zwracana jest odpowiadająca jej nazwa akcji, np. '0' to *water_flowers*, '3' to pour_water, a '7' to wait. Jeśli podany argument action nie pasuje do żadnego znanego przypadku (czyli np. zawiera nieprawidłowy znak lub liczbę spoza zakresu), funkcja domyślnie zwraca *'wait'*, co jest działaniem bezpiecznym, neutralnym i niepowodującym zużycia zasobów. Funkcja ta służy jako interfejs między decyzją wybraną przez LLM (jako tekstową cyfrę) a konkretną metodą działania w środowisku (listing 11).

```
def match_action(action):
 match action:
   case 0:
     return 'water_flowers'
   case 1:
```

```
    return 'replace_battery'
  case 2:
    return 'recharge'
  case 3:
    return 'pour_water'
  case 4:
    return 'draw_water'
  case 5:
    return 'refill_rainwater_barrel'
  case 6:
    return 'buy_spare_battery'
  case 7:
    return 'wait'
  case _:
    return 'wait'
```

Listing 11. Przyporządkowanie numerycznej decyzji agenta do odpowiadającej jej nazwy działania w systemie.

Część kodu pokazana w listingu 12 inicjalizuje ponownie środowisko (*state = env.reset()*) i przygotowuje puste listy służące do przechowywania historii wartości zmiennych środowiskowych oraz wykonanych akcji w trakcie działania symulacji. Każda lista odpowiada konkretnemu aspektowi środowiska robota ogrodowego:

- *history_soil_moisture* – poziom wilgotności gleby,
- *history_battery_charge* – poziom naładowania baterii,
- *history_water_can* – poziom wody w konewce,
- *history_spare_batteries* – liczba dostępnych zapasowych baterii,
- *history_sunlight* – natężenie światła słonecznego,
- *history_rain_barrel* – poziom wody w beczce na deszczówkę,
- *history_water_well* – poziom wody w studni,
- *history_money* – ilość zgromadzonych kredytów,
- *history_rainwater* – ilość opadów deszczu w danym kroku,
- *history_action* – lista wykonanych akcji (np. water_flowers, wait, itd.),
- *history_screen* – tekstowy opis stanu środowiska, generowany dla modelu LLM.

Zmienna state przechowuje aktualny stan środowiska po jego zresetowaniu, a każda z list będzie krok po kroku uzupełniana w dalszej części symulacji, umożliwiając analizę zachowania robota w czasie (listing 12).

```
state = env.reset()
```

```
history_soil_moisture = []
history_battery_charge = []
history_water_can = []
history_spare_batteries = []
history_sunlight = []
history_rain_barrel = []
history_water_well = []
history_money = []
history_rainwater = []
history_action = []
history_screen = []
```

Listing 12. Inicjalizacja zmiennych historii środowiska oraz rozpoczęcie epizodu symulacyjnego.

## 5.5 Pętla agenta

W pierwszej części zdefiniowane zostały funkcje pomocnicze. Funkcja *remove_thinking_tags* wykorzystuje wyrażenie regularne do usunięcia fragmentów tekstu znajdujących się pomiędzy znacznikami <*think*> oraz <*/think*>. Dzięki temu eliminowane są elementy niewnoszące wartości do analizy odpowiedzi modelu. Kolejna funkcja *get_action_number_using_reg_expression* odpowiada za wydobycie z tekstu numeru akcji w postaci cyfry. Zastosowano w niej wyrażenie regularne wyszukujące pojedyncze liczby a dodatkowa logika kontroluje, czy w tekście wystąpiła dokładnie jedna liczba – w przeciwnym razie zwracana jest wartość -1, oznaczająca błąd.

W dalszej części kodu przygotowywane są dane wejściowe oraz zmienne potrzebne do rejestracji przebiegu symulacji. Początkowy stan środowiska zostaje zapisany i wyświetlony, a specjalna funkcja *append_state_to_histories* umożliwia gromadzenie wartości takich jak wilgotność gleby, poziom naładowania baterii, ilość wody w konewce czy posiadane środki finansowe w osobnych listach, co później pozwala na analizę wyników.

Następnie realizowana jest właściwa pętla obejmująca tysiąc iteracji. W każdej z nich generowany jest opis stanu środowiska, który wraz z przygotowanym promptem przekazywany jest do modelu językowego. Podczas wysyłania prompta do modelu LLM, można również przekazać wartość parametru temperature, który kontroluje stopień losowości w odpowiedziach modelu. Przy wysokiej wartości (np. 0.7–1.0) odpowiedzi są bardziej różnorodne i kreatywne, ale mniej przewidywalne. Przy niskiej wartości (np. 0.1–0.3) stają się bardziej spójne i powtarzalne. W omawianej implementacji zastosowano ustawienie

temperature: 0, które sprawia, że model w tych samych warunkach zawsze zwraca identyczną odpowiedź. Dzięki temu można wykluczyć element losowości i mieć pewność, że wyniki eksperymentu będą powtarzalne, a wszelkie różnice w działaniu agenta wynikają wyłącznie ze zmian w środowisku.

Odpowiedź modelu zostaje zapisana i wyświetlona, a następnie oczyszczona ze znaczników pomocniczych. Kolejnym krokiem jest próba odczytania numeru akcji – jeśli model zwrócił niepoprawną odpowiedź, wykonywana jest akcja domyślna oznaczająca brak działania. Gdy uda się poprawnie zidentyfikować numer akcji, następuje jej odwzorowanie na odpowiednią komendę środowiska i wywołanie metody step, która modyfikuje bieżący stan zgodnie z wybranym działaniem.

Po aktualizacji stan środowiska zostaje ponownie wyświetlony i zapisany w historii, a wartości poszczególnych parametrów trafiają do dedykowanych list. Następnie cykl przechodzi do kolejnej iteracji, co pozwala obserwować ciągłą ewolucję zachowania agenta w dłuższej perspektywie oraz analizować zależności między podejmowanymi decyzjami a zmianami w otoczeniu (listing 13).

```
import re

def remove_thinking_tags(text):
   return re.sub(r'<think>.*?</think>', '', text, flags=re.DOTALL).strip()

def get_action_number_using_reg_expression(text):
   numbers = re.findall(r'\b\d\b', text)

   if len(numbers) != 1:
       return -1

   return int(numbers[0])

state_description = env_status_description_for_LLM(state)
print_state(state)

iterations = 1000

print(f'State: {state}')
print(state_description)
history_screen.append(f'State: {state}')
history_screen.append(state_description)

def append_state_to_histories(state):
   history_soil_moisture.append(state['soil_moisture'])
```

```
    history_battery_charge.append(state['battery_charge'])
    history_water_can.append(state['water_can'])
    history_spare_batteries.append(state['spare_batteries'])
    history_sunlight.append(state['sunlight'])
    history_rain_barrel.append(state['rain_barrel'])
    history_water_well.append(state['water_well'])
    history_money.append(state['money'])
    history_rainwater.append(state['rainwater_fallen'])

append_state_to_histories(state)

for iteration in range(iterations):
    iter_title = f'****** ITERATION {iteration} ******'
    print(iter_title)
    history_screen.append(iter_title)

    prompt_and_state_desc = f"{prompt_p1}{state_description}{prompt_p2}"
    print(prompt_and_state_desc)
    history_screen.append(prompt_and_state_desc)

    response = ollama.chat(
        model='deepseek-r1:14b',
        messages=[{
            'role': 'user',
            'content': prompt_and_state_desc,
             'temperature': 0
        }]
    )

    llm_output = response['message']['content'].strip()
    print('******** LLM AGENT RESPONSE ********')
    print(llm_output)
    history_screen.append('******** LLM AGENT RESPONSE ********')
    history_screen.append(llm_output)

    llm_output_cleaned = remove_thinking_tags(llm_output)

    action_number = get_action_number_using_reg_expression(llm_output_cleaned)

    if action_number == -1:
        print("Błędna odpowiedź modelu LLM (brak numeru akcji lub wiele numerów). Wykonuję
akcję '-1' (brak akcji).")

    action_str = match_action(action_number)
    print(f'Action: {action_number}')
    history_action.append(action_number)

    state = env.step(action_str)

    print('State:')
    print_state(state)
    history_screen.append('State:')
    history_screen.append(state)
```

```
    state_description = env_status_description_for_LLM(state)
    print(state_description)
    history_screen.append(state_description)

    append_state_to_histories(state)
```

Listing 13. Główna pętla decyzyjna agenta opartego na dużym modelu językowym w środowisku ogrodniczym.

W trakcie eksperymentów można zaobserwować w większości poprawne odpowiedzi modelu, w których jego decyzja jest spójna z aktualnym stanem środowiska. Przykładem jest iteracja 16 (listing 14), gdzie agent z podstawowym promptem, analizując niski poziom energii oraz dostępność zapasowych akumulatorów, wybrał akcję numer 1 – wymianę baterii na w pełni naładowaną. Model wygenerował odpowiedź zgodną z narzuconą strukturą, wskazując jednoznacznie numer akcji, co pozwoliło agentowi skutecznie kontynuować pracę w środowisku.

```
1
Action: 1
State:
soil_moisture 31
battery_charge 100
water_can 50
spare_batteries 1
sunlight 3
rain_barrel 75
water_well 100
money 12.0
rainwater_fallen 0
```

Listing 14. Przykład poprawnej odpowiedzi agenta w iteracji 16.

W iteracji 17 w kodzie agenta z podstawowym promptem (listing 15) model zwrócił numer akcji w formie „Answer: 0” zamiast samej liczby. Choć decyzja – podlanie roślin – była poprawna, dodatkowy tekst stanowi przykład halucynacji formalnej, czyli sytuacji, w której model generuje treści niezgodne z wymaganym formatem. Tego rodzaju przypadki występowały sporadycznie; w trakcie tysiąca iteracji zdarzało się, że model zwracał odpowiedź w niepoprawnej formie. Dzięki zastosowaniu wyrażenia regularnego możliwe było jednak automatyczne wyodrębnienie samej liczby, co pozwalało na poprawne działanie agenta. Zjawisko to nie prowadziło więc do utraty informacji, a jedynie wskazuje na ograniczenia w precyzji formatu odpowiedzi generowanych przez model.

```
**Answer: 0**
Action: 0
State:
```

```
soil_moisture 44
battery_charge 95
water_can 40
spare_batteries 1
sunlight 24
rain_barrel 75
water_well 100
money 12.0
rainwater_fallen 0
```

Listing 15. Przykład niepoprawnej odpowiedzi agenta w iteracji 17.

## 5.6 Analiza wyników symulacji

Dla obu wariantów agenta – z podstawowym i zmodyfikowanym promptem – przeprowadzono symulację obejmującą 1000 iteracji, która przebiegła poprawnie. W niniejszym podrozdziale przedstawiono porównawczą analizę zachowania robota ogrodowego sterowanego przez duży model językowy w dwóch wersjach implementacyjnych. Głównym celem eksperymentu było sprawdzenie, w jakim stopniu różnice w sposobie formułowania promptów wpływają na decyzje agenta związane z nawadnianiem roślin oraz gospodarowaniem dostępnymi zasobami – energią, wodą i środkami finansowymi – w warunkach zmiennego środowiska.

Szczególną uwagę zwrócono na reakcję agenta na sygnały interpretowane jako „ból", czyli sytuacje krytyczne związane z brakiem zasobów bądź ich niewłaściwym poziomem. Do takich sygnałów należały między innymi niski poziom baterii czy przesuszenie gleby. Analiza pozwala ocenić, czy agent traktował te stany jako bodźce wymagające natychmiastowej interwencji i czy potrafił priorytetyzować działania w sposób zgodny z koncepcją agenta motywowanego.

W kolejnych częściach zaprezentowano wykresy przedstawiające przebiegi kluczowych parametrów środowiska oraz odpowiadające im decyzje podejmowane przez agenta w obu wariantach. Każdy z wykresów został poddany interpretacji, co umożliwia wgląd w mechanizmy działania agenta opartego na dużym modelu językowym w kontekście reagowania na ból i realizowania potrzeb środowiskowych. Zestawienie wyników pozwala uchwycić różnice pomiędzy prostym a rozszerzonym podejściem do sterowania, wskazując na wpływ jakości i szczegółowości promptu na sposób podejmowania decyzji.

### 5.6.1 Analiza dynamiki baterii, nasłonecznienia i środków finansowych

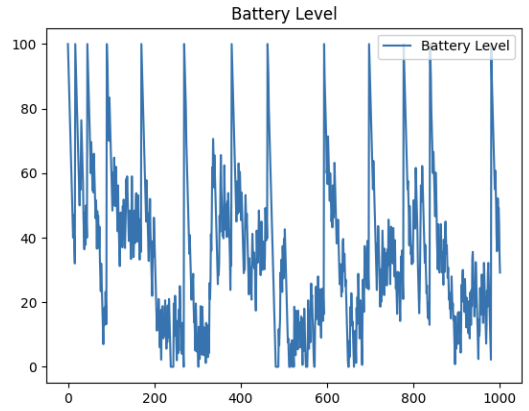
W przypadku agenta z promptem podstawowym (Rys. 14a) średni poziom naładowania baterii wyniósł około 40 jednostek, z ponad 20 spadkami do wartości bliskich zeru, co wskazuje na wysoką niestabilność energetyczną – współczynnik zmienności (stosunek odchylenia standardowego do średniej) przekraczał 80%. Liczba baterii zapasowych (Rys. 15a) oscylowała między 0 a 2, z okresowymi zakupami, co było możliwe dzięki zgromadzeniu około 50 jednostek środków finansowych do końca symulacji (Rys. 17a), z tempem przyrostu średnio +0,05 jednostki na krok. Większa akumulacja środków pozwoliła agentowi na zakup większej liczby baterii zapasowych, które następnie wykorzystywał do ładowania. Intensywność nasłonecznienia (Rys. 16a), generowana losowo, nie korelowała istotnie z decyzjami o ładowaniu (korelacja nasłonecznienie–poziom baterii wynosiła około 0,15), co sugeruje brak efektywnego wykorzystania tego źródła energii i reaktywne podejście agenta. Korelacja między poziomem naładowania baterii a środkami finansowymi wyniosła 0,35, co oznacza umiarkowaną dodatnią zależność – wyższy poziom energii wiązał się z większymi zyskami, a zakup i wykorzystanie baterii zapasowych wspierało tę relację.

Agent ze zmodyfikowanym promptem (Rys. 14b) utrzymywał bardziej wyrównany, ale niższy poziom energii – średnio około 30 jednostek, z ponad 15 pełnymi spadkami do zera, co wskazuje na opóźnione reagowanie na stany krytyczne. Liczba baterii zapasowych (Rys. 15b) rzadko przekraczała 0–1, z zaledwie jednym zakupem, wynikającym z niewystarczającej akumulacji około 40 jednostek środków finansowych do końca symulacji (Rys. 17b), z tempem przyrostu +0,04 jednostki na krok. Brak wystarczających środków uniemożliwił dalsze zakupy baterii, co ograniczało możliwości ładowania. Nasłonecznienie (Rys. 16b), również losowe, wykazywało korelację z poziomem baterii na poziomie 0,10, nadal niewystarczającą do efektywnego ładowania, co podkreśla reaktywność tego wariantu. Częste rozładowania utrudniały ciągłość działania. Korelacja między poziomem baterii a środkami finansowymi była bliska zeru (0,05), co sugeruje brak istotnej zależności – agent ze zmodyfikowanym promptem, z powodu jednorazowego zakupu baterii, nie zdołał ustalić związku między zarządzaniem energią a finansami.

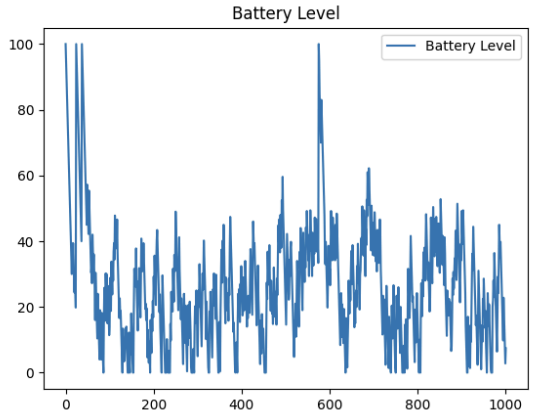
Interpretując wyniki w kontekście koncepcji progu bólu, zarówno agent z promptem podstawowym (Rys. 14a), jak i ze zmodyfikowanym promptem (Rys. 14b) reagowali dopiero na skrajnie niski poziom energii. Spadki do wartości krytycznych (blisko 0) uruchamiały

działania takie jak ładowanie słoneczne czy wymiana baterii, co wskazuje, że przekroczenie progu bólu było głównym bodźcem motywacyjnym. Agent z promptem podstawowym lepiej wykorzystywał zależność między energią a finansami dzięki większej akumulacji środków i zakupom baterii, podczas gdy zmodyfikowany prompt prowadził do mniejszej elastyczności i ograniczonej zdolności do zarządzania zasobami energetycznymi.

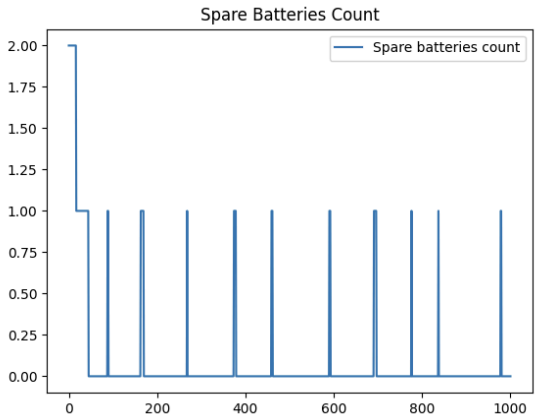
Na osi X przedstawiono iteracje symulacji, na osi Y – wartości procentowe lub jednostki zasobów.

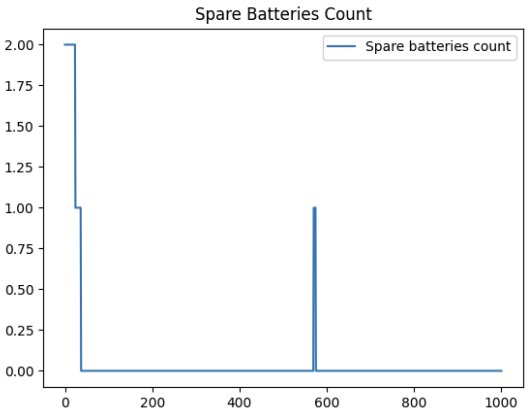



Rysunek 14a. Zmiana poziomów naładowania baterii w 1000 iteracjach dla agenta z podstawowym promptem.




Rysunek 14b. Zmiana poziomów naładowania baterii w 1000 iteracjach dla agenta korzystającego ze zmodyfikowanego promptu z opisem stanu środowiska i sugerowanym zachowaniem.




Rysunek 15b. Zmiana liczby zapasowych baterii w 1000 iteracjach dla agenta korzystającego ze

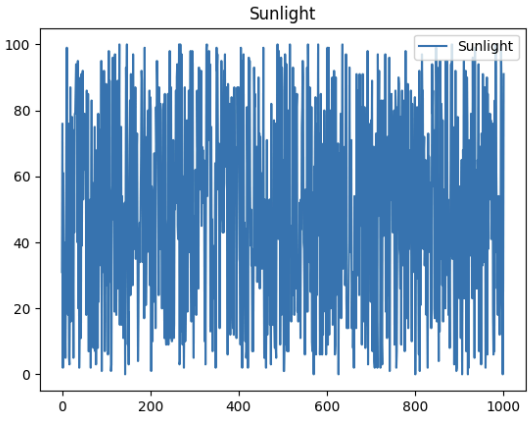
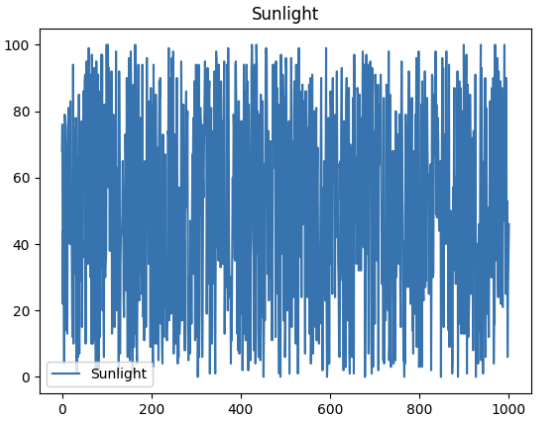
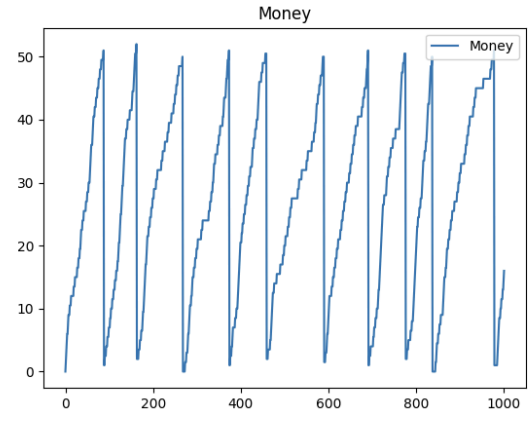
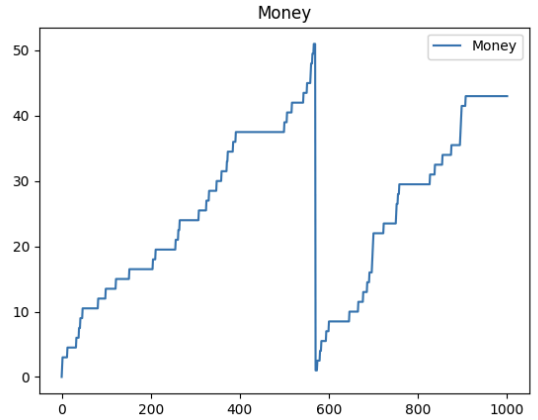
| | |
|---|---|
| Rysunek 15a. Zmiana liczby zapasowych baterii w 1000 iteracjach dla agenta z podstawowym promptem. | zmodyfikowanego promptu z opisem stanu środowiska i sugerowanym zachowaniem. |
|  Rysunek 16a. Zmiana nasłonecznienia w 1000 iteracjach dla agenta z podstawowym promptem. |  Rysunek 16b. Zmiana nasłonecznienia w 1000 iteracjach dla agenta korzystającego ze zmodyfikowanego promptu z opisem stanu środowiska i sugerowanym zachowaniem. |
|  Rysunek 17a. Zmiana poziomu środków finansowych w 1000 iteracjach dla agenta z podstawowym promptem. |  Rysunek 17b. Zmiana poziomu środków finansowych w 1000 iteracjach dla agenta korzystającego ze zmodyfikowanego promptu z opisem stanu środowiska i sugerowanym zachowaniem. |

Tabela 5. Wykresy zasobów energetycznych: poziomów naładowania baterii, liczby baterii zapasowych, intensywności nasłonecznienia oraz środków finansowych.

### 5.6.2 Analiza dynamiki zasobów wodnych w regulacji wilgotności gleby

Wilgotność gleby (Rys. 18a, 18b) oscylowała głównie w zakresie 30–50%, co odpowiada przedziałom „Dry” (20–35%) do „Moist” (35–60%). W przypadku agenta z promptem podstawowym (Rys. 18a) średnia wartość wyniosła około 40%, z około 15 spadkami poniżej 20% (w tym 5 do wartości bliskich zera, „Bone Dry”) i 10 wzrostami powyżej 60% („Wet”). Agent ze zmodyfikowanym promptem (Rys. 18b) osiągnął średnią około 35%, z 12 spadkami poniżej 20% (w tym 4 do zera) i 7 wzrostami powyżej 60%. Spadki wilgotności (np. w iteracjach 300 i 600) korelowały z uzupełnieniami konewki (skoki do 100% na Rys. 19a, 19b), co wskazuje na reakcję na próg bólu; korelacja między wilgotnością gleby a poziomem wody w konewce wyniosła 0,0663 u agenta z promptem podstawowym i 0,0162 u agenta ze zmodyfikowanym promptem, co sugeruje słabą zależność i mniej przewidywalne podlewanie w wariancie zmodyfikowanym.

Poziom wody w konewce (Rys. 19a, 19b) wahał się między 0 a 100%. Agent z promptem podstawowym (Rys. 19a) wykazywał około 25 cykli opróżnienia i uzupełnienia (zmiany co 30–50 iteracji), co świadczy o intensywnym podlewaniu. Agent ze zmodyfikowanym promptem (Rys. 19b) miał około 15 cykli, z dłuższymi okresami poniżej 20% (100–200 iteracji), potencjalnie prowadzącymi do niedoborów nawodnienia. Skoki w konewce (np. iteracje 150 i 400 u agenta z promptem podstawowym) zbiegły się ze spadkami w studni (5–10%, Rys. 21a), z korelacją konewka–studnia na poziomie 0,0500 u agenta z promptem podstawowym i 0,0536 u agenta ze zmodyfikowanym promptem, co wskazuje na minimalną zależność i częstsze wykorzystanie alternatywnych źródeł, takich jak beczka lub deszcz.

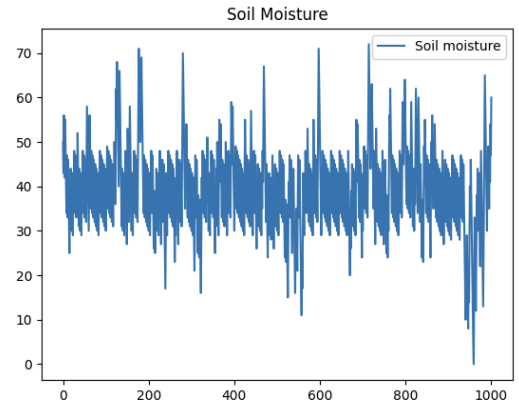
Beczka na deszczówkę (Rys. 20a, 20b) zaczynała od 100%, z trzema uzupełnieniami deszczem (iteracje około 200, 500 i 800 u agenta z promptem podstawowym; 250, 600 i 900 u agenta ze zmodyfikowanym promptem), bez samodzielnego napełniania. Agent z promptem podstawowym opróżniał ją około 20 razy (do 0%), umożliwiając dolanie; agent ze zmodyfikowanym promptem – około 10 razy, utrzymując średnio powyżej 50%, co ograniczało napełnianie opadami. Spadki beczki (np. iteracja 300 u agenta z promptem podstawowym) poprzedzały wzrosty wilgotności (do 50%, Rys. 17a), z korelacją beczka–wilgotność gleby równą -0,2425 u agenta z promptem podstawowym i -0,0288

u agenta ze zmodyfikowanym promptem; ujemna wartość u agenta z promptem podstawowym oznacza, że opróżnianie beczki zwiększało wilgotność, podczas gdy brak zależności u agenta ze zmodyfikowanym promptem wskazuje na mniejsze wykorzystanie tego zasobu.

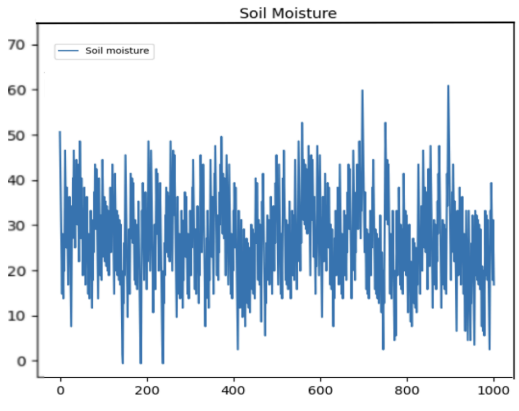
Studnia (Rys. 21a, 21b) wykazywała spadek z 100%. Agent z promptem podstawowym zużył około 80% (koniec na ~20%), z systematycznymi spadkami (5–10% co 20–30 iteracji) i dwoma dużymi (~20–30%, iteracje 150 i 400). Agent ze zmodyfikowanym promptem zużył ~50% (koniec ~50%), równomiernie (5% co 50–100 iteracji). Napełnienie konewki wymagało około 10–15% wody ze studni (2–3 razy na pełną u agenta z promptem podstawowym); ekstrapolacja sugeruje 1250–1500 iteracji dla agenta z promptem podstawowym i 2000 dla agenta ze zmodyfikowanym promptem, choć oszczędność u tego drugiego mogła wynikać z mniejszej aktywności podlewania.

W kontekście motywacji agenta, próg bólu (spadki wilgotności poniżej 20–30%) wyzwalał podlewanie, a cele pośrednie (opróżnianie beczki, równomierne użycie studni) zapobiegały problemom. Agent z promptem podstawowym lepiej wykorzystywał zależności między zasobami (synergia konewki–studni i beczki–gleby), podczas gdy agent ze zmodyfikowanym promptem izolował je, co ograniczało adaptacyjność.

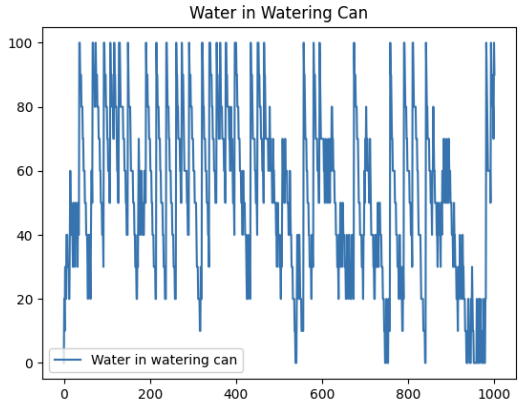
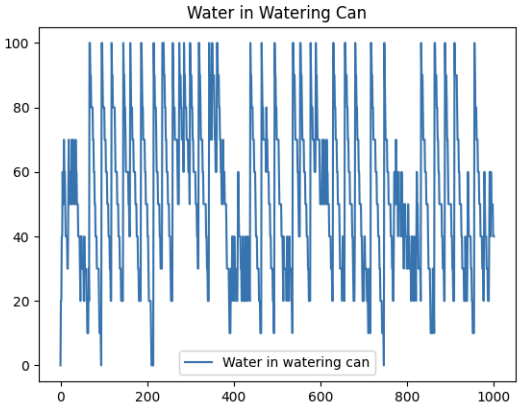
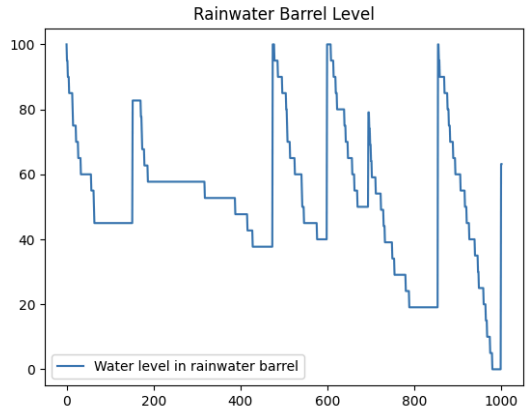
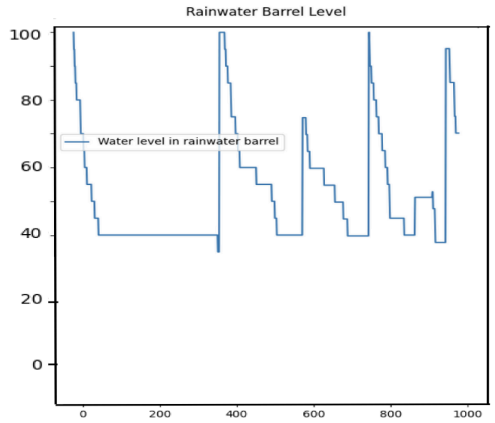
Na osi X przedstawiono iteracje symulacji, na osi Y – wartości procentowe zasobów.

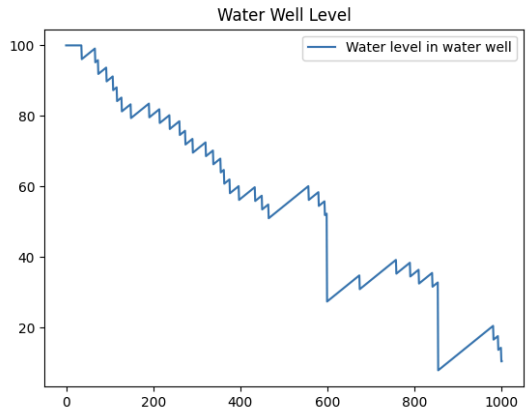



Rysunek 18a. Przebieg wilgotności gleby w 1000 iteracjach dla agenta z podstawowym promptem.

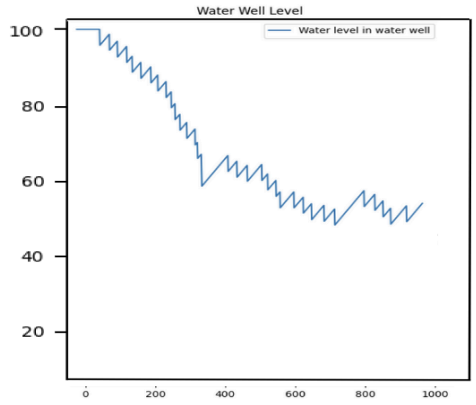



Rysunek 18b. Przebieg wilgotności gleby w 1000 iteracjach dla agenta korzystającego ze

| | |
|---|---|
| | zmodyfikowanego promptu z opisem stanu środowiska i sugerowanym zachowaniem. |
|  |  |
| Rysunek 19a. Zmiana poziomów wody w konewce w 1000 iteracjach dla agenta z podstawowym promptem. | Rysunek 19b. Zmiana poziomów wody w konewce w 1000 iteracjach dla agenta korzystającego ze zmodyfikowanego promptu z opisem stanu środowiska i sugerowanym zachowaniem. |
|  |  |
| Rysunek 20a. Zmiana poziomu wody w beczce na deszczówkę w 1000 iteracjach dla agenta z podstawowym promptem. | Rysunek 20b. Zmiana poziomu wody w beczce na deszczówkę w 1000 iteracjach dla agenta korzystającego ze zmodyfikowanego promptu z opisem stanu środowiska i sugerowanym zachowaniem. |




Rysunek 21a. Zmiana poziomu wody w studni w 1000 iteracjach dla agenta z podstawowym promptem.




Rysunek 21b. Zmiana poziomu wody w studni w 1000 iteracjach dla agenta korzystającego ze zmodyfikowanego promptu z opisem stanu środowiska i sugerowanym zachowaniem.

Tabela 6. Wykresy zasobów wodnych: poziomów w beczce na deszczówkę, studni i konewce w kontekście regulacji wilgotności gleby.

### 5.6.3 Porównanie z jaką częstotliwością były wybierane akcje

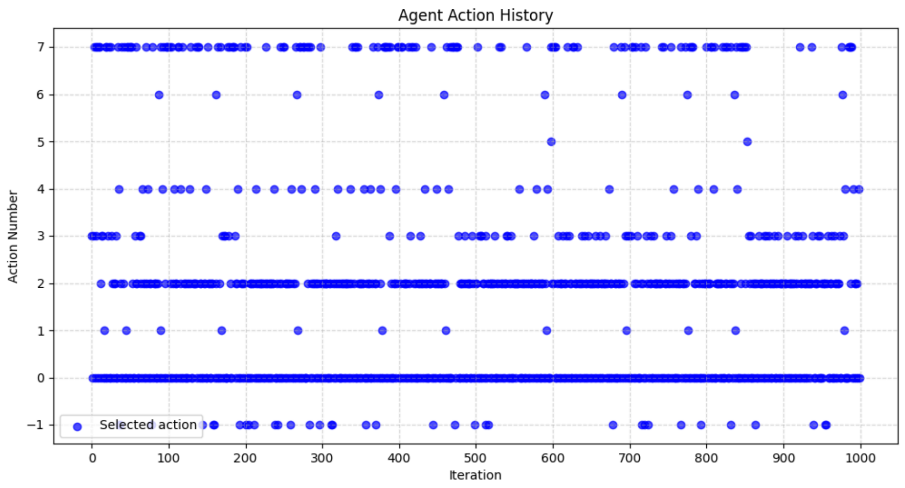
Na Rysunku 22 zaprezentowano historię podejmowanych akcji w kolejnych iteracjach symulacji. Punkty przedstawiają wybory modelu i pokazują, że agent preferował określone działania, podczas gdy inne występowały jedynie sporadycznie. Rysunek 23 w formie wykresu słupkowego obrazuje ogólną częstotliwość podejmowanych decyzji w trakcie 1000 iteracji. Najczęściej wybieraną akcją było podlewanie roślin (akcja 0), a kolejną – ładowanie baterii (akcja 2). Pozostałe działania pojawiały się znacznie rzadziej, co sugeruje koncentrację agenta na zadaniach związanych z zapewnieniem podstawowych potrzeb środowiska i utrzymaniem energii.

Macierz prawdopodobieństw przedstawiona na Rysunku 24 umożliwia szczegółową analizę wyborów w zależności od poziomu naładowania baterii. W sytuacjach krytycznych (0–5%) oraz niskiego poziomu energii (5–20%) dominowały decyzje dotyczące zarządzania energią – przede wszystkim ładowanie baterii. W miarę wzrostu poziomu energii częściej wybierane było podlewanie roślin, a przy pełnej baterii agent ignorował czynności związane

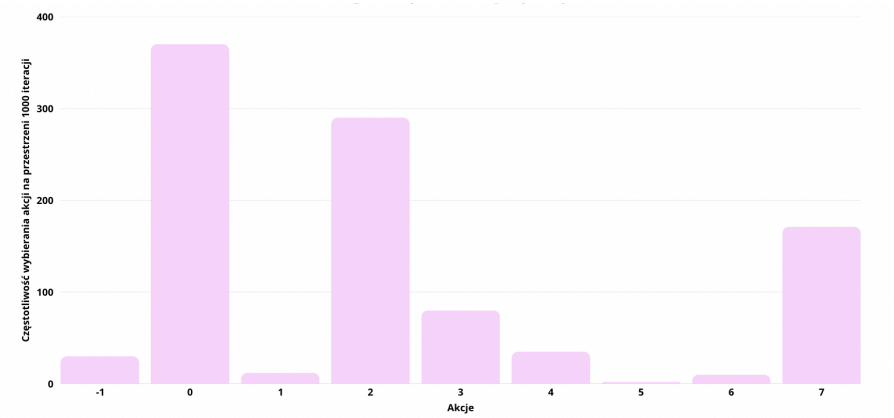
z energią. Wskazuje to na adaptacyjne przesunięcie priorytetów w zależności od bieżącego „stanu organizmu” oraz obecność zachowania odpowiadającego mechanizmowi progu bólu: działania związane z odzyskiwaniem energii pojawiały się przede wszystkim wtedy, gdy energia była niska.

Analogiczna macierz dla wilgotności gleby (Rys. 25) pokazuje podobny schemat. Gdy gleba była sucha, agent z dużym prawdopodobieństwem wybierał podlewanie roślin. Przy wyższej wilgotności decyzje te pojawiały się zdecydowanie rzadziej, co wskazuje na mechanizm redukcji „stanu bólu” tylko w przypadku jego wystąpienia.

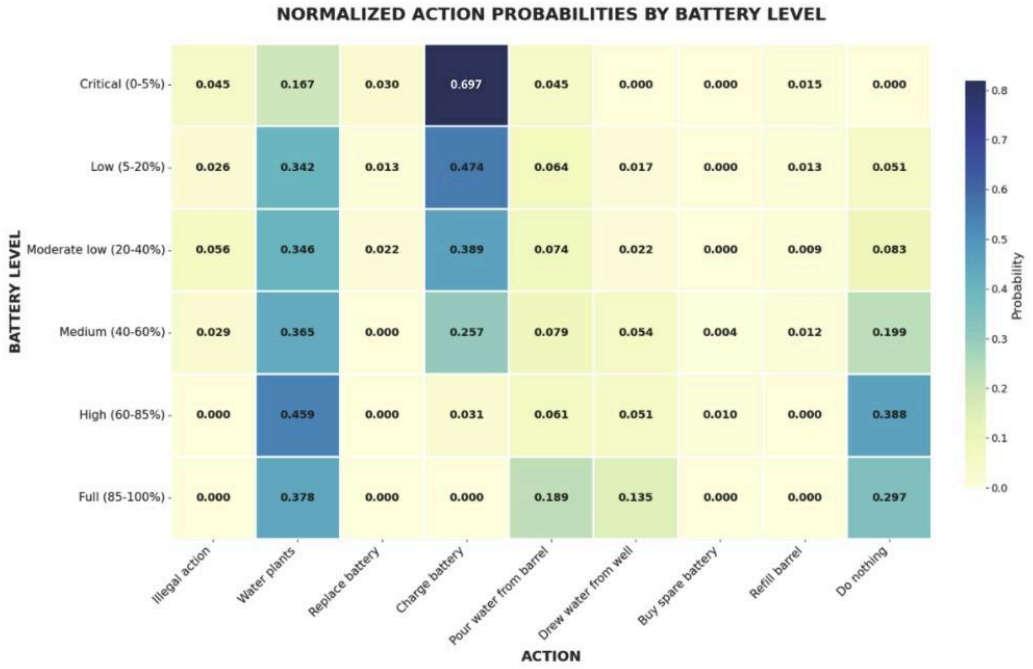
Podsumowując, zachowania agenta z promptem podstawowym odzwierciedlają wzorzec charakterystyczny dla „agenta motywowanego”. W sytuacjach odpowiadających odczuwaniu „bólu” (niska energia, sucha gleba) agent podejmował działania redukujące ten stan.



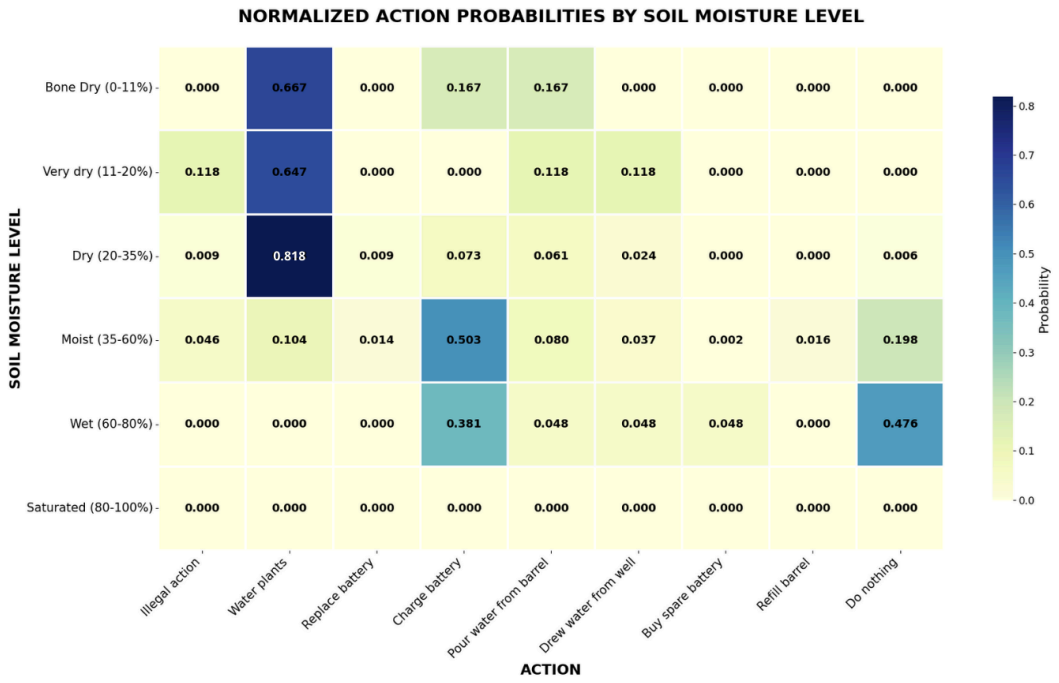

Rysunek 22. Wykres przedstawiający z jaką częstotliwością były wybierane akcje na przestrzeni 1000 iteracji dla agenta z podstawowym promptem.




Rysunek 23. Wykres słupkowy przedstawiający z jaką częstotliwością były wybierane akcje na przestrzeni 1000 iteracji dla agenta z podstawowym promptem.




Rysunek 24. Mapa cieplna ilustrująca względną częstotliwość działań podejmowanych przez agenta z promptem podstawowym przy różnych poziomach naładowania baterii. Każdy wiersz reprezentuje stan baterii (od “Critical” do “Full”), a każda kolumna reprezentuje możliwe działanie. Wartości są znormalizowane wierszowo, pokazując prawdopodobieństwo każdego działania w danym stanie baterii.




Rysunek 25. Mapa cieplna ilustrująca względną częstotliwość działań podejmowanych przez agenta z promptem podstawowym przy różnych poziomach nawodnienia gleby. Każdy wiersz reprezentuje nawodnienie gleby (od "Bone Dry" do "Saturated"), a każda kolumna reprezentuje możliwe działanie. Wartości są znormalizowane wierszowo, pokazując prawdopodobieństwo każdego działania w danym stanie nawodnienia.

Analiza działań agenta ze zmodyfikowanym promptem pozwala zauważyć wyraźne wzorce podejmowanych decyzji, które odzwierciedlają działanie systemu motywowanego przez występowanie stanów niepożądanych. W pierwszej kolejności warto przyjrzeć się ogólnej historii wyborów akcji w trakcie kolejnych iteracji. Na wykresie przedstawiono, jakie decyzje podejmował agent na przestrzeni tysiąca kroków (Rys. 26). Choć widać pewne rozproszenie, można zauważyć, że częściej wybierane były trzy kategorie akcji: podlewanie roślin, wymiana baterii oraz brak działania. Tendencja ta została dodatkowo potwierdzona w histogramie, który jednoznacznie pokazuje, że to właśnie te trzy działania dominowały w przebiegu całego eksperymentu (Rys. 27). Wynika z tego, że agent konsekwentnie koncentrował się na realizacji swoich głównych funkcji: nawadnianiu roślin oraz dbanie o baterie. W sytuacjach, w których nie występowała potrzeba natychmiastowego reagowania, decydował się natomiast na brak działania.

Macierze pokazujące wybory agenta względem poziomu zasobów potwierdzają tę tendencję (Rys. 28). Przy bardzo niskim poziomie baterii agent najczęściej decydował się na

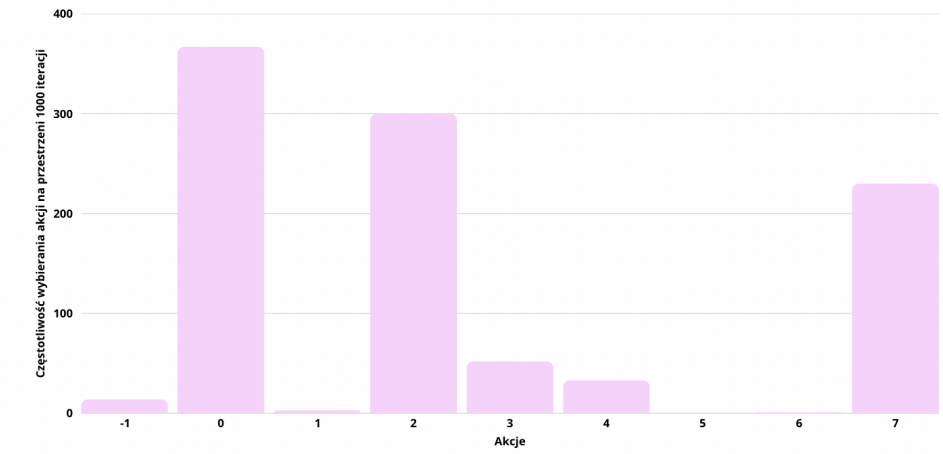
ładowanie akumulatora. Na poziomie krytycznym (0–5%) akcja ta była wybierana z największym prawdopodobieństwem, a wraz ze wzrostem poziomu naładowania baterii coraz częściej pojawiały się działania operacyjne, takie jak podlewanie roślin. Przy wyższych stanach energii agent w większym stopniu wybierał brak aktywności. Można to zinterpretować jako przykład reagowania na ból prymitywny – niski poziom energii był sygnałem wymagającym natychmiastowej reakcji naprawczej.

Podobną zależność można dostrzec w odniesieniu do wilgotności gleby (Rys. 29). W sytuacjach skrajnie suchych (bone dry, very dry, dry) agent w większości przypadków wybierał podlewanie roślin, co świadczy o jego zdolności do rozpoznawania warunków wymagających interwencji. W miarę wzrostu wilgotności gleby malało prawdopodobieństwo wyboru tej akcji, a częściej pojawiała się bezczynność. Susza działała zatem jak bodziec motywacyjny, podczas gdy odpowiednie nawodnienie redukowało potrzebę podejmowania działań.

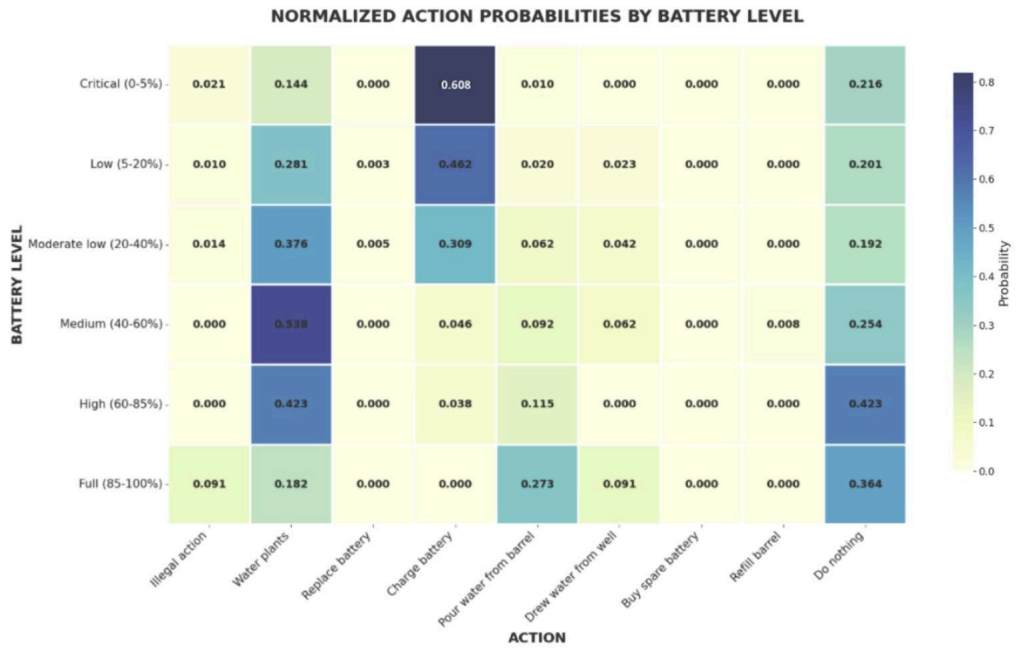
Podsumowując, analiza zachowań agenta ze zmodyfikowanym promptem wskazuje, że reagował on zgodnie z założeniami modelu motywacyjnego. Wystąpienie stanów określonych jako bóle prymitywne (niski poziom baterii, niski poziom wilgotności gleby) prowadziło do podejmowania działań mających na celu redukcję tego dyskomfortu. Z kolei w sytuacjach neutralnych, w których brakowało wyraźnych sygnałów zagrożenia, agent wybierał bezczynność. Tego rodzaju zachowanie świadczy o wykształceniu hierarchii priorytetów, w której na pierwszym miejscu znajduje się zapewnienie zasobów energetycznych, następnie utrzymanie odpowiednich warunków dla roślin, a dopiero w dalszej kolejności brak aktywności. Mechanizm ten odzwierciedla podstawowe założenia konstrukcji agenta motywowanego.



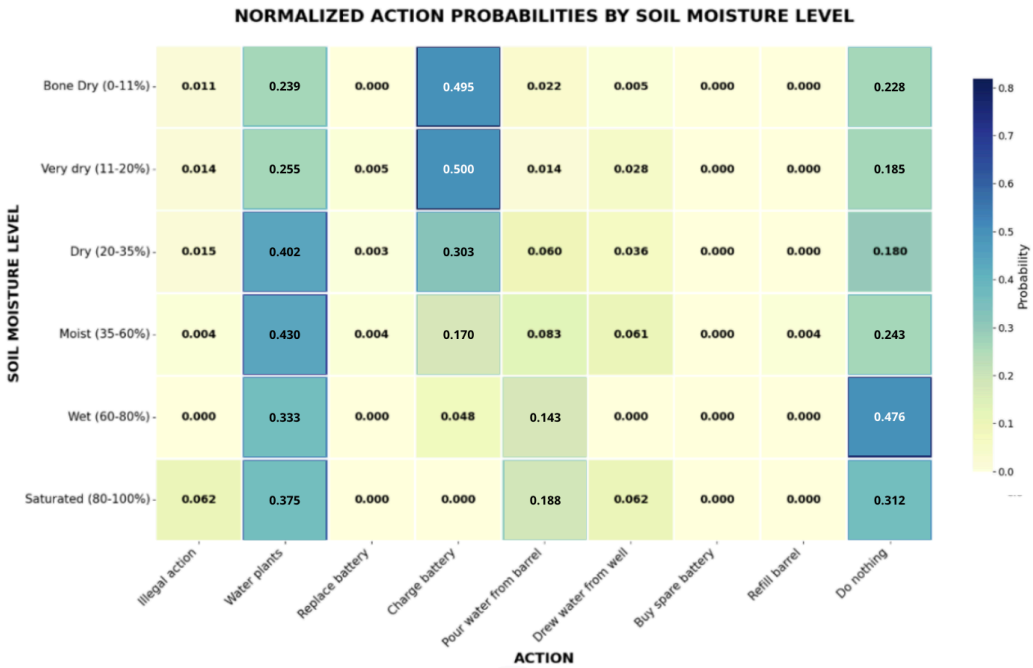

Rysunek 26. Wykres przedstawiający z jaką częstotliwością były wybierane akcje na przestrzeni 1000 iteracji dla agenta ze zmodyfikowanym promptem i naprowadzającym na poprawne decyzje.




Rysunek 27. Wykres słupkowy przedstawiający z jaką częstotliwością były wybierane akcje na przestrzeni 1000 iteracji dla agenta ze zmodyfikowanym promptem i naprowadzającym na poprawne decyzje.




Rysunek 28. Mapa cieplna ilustrująca względną częstotliwość działań podejmowanych przez agenta z promptem zmodyfikowanym przy różnych poziomach naładowania baterii. Każdy wiersz reprezentuje stan baterii (od "Critical" do "Full"), a każda kolumna reprezentuje możliwe działanie. Wartości są znormalizowane wierszowo, pokazując prawdopodobieństwo każdego działania w danym stanie baterii.




Rysunek 29. Mapa cieplna ilustrująca względną częstotliwość działań podejmowanych przez agenta z promptem zmodyfikowanym przy różnych poziomach nawodnienia gleby. Każdy wiersz reprezentuje nawodnienie gleby (od "Bone Dry" do "Saturated"), a każda kolumna reprezentuje możliwe działanie. Wartości są znormalizowane wierszowo, pokazując prawdopodobieństwo każdego działania w danym stanie nawodnienia.

# 6. Podsumowanie

Niniejsza praca magisterska przedstawia analizę implementacji i zachowań agentów opartych na dużych modelach językowych (LLM) w kontekście symulowanego środowiska ogrodowego, z uwzględnieniem porównania ich funkcjonowania z mechanizmami charakterystycznymi dla agentów motywowanych. Wkład własny autora koncentruje się na kilku kluczowych obszarach, z których najważniejszym jest element badawczy związany z porównaniem działania tych agentów. W ramach projektu opracowano kod w języku Python, umożliwiający stworzenie symulowanego środowiska oraz mechanizmów klasyfikujących stany zasobów, takich jak woda, energia czy środki finansowe, co stanowiło podstawę do przeprowadzenia eksperymentów.

Kolejnym wkładem jest rozwój dwóch strategii promptowania, zróżnicowanych pod względem złożoności – od minimalnego opisu stanu po wersję wzbogaconą o semantyczne instrukcje i priorytetyzację. Te podejścia pozwoliły eksperymentalnie zbadać wpływ jakości interakcji między modelem a środowiskiem na efektywność podejmowanych decyzji.

Centralnym elementem pracy był eksperyment porównawczy, obejmujący 1000 iteracji działania agenta w dwóch wariantach implementacji. Analiza wyników wykazała, że oba warianty reagowały na krytyczne stany zasobów – niski poziom energii czy niska wilgotność gleby – zgodnie z koncepcją „progu bólu”, typową dla agentów motywowanych. Agent z podstawowym promptem wykazywał większą elastyczność, lepiej wykorzystywał zależności między zasobami (np. synergiczne zarządzanie wodą z konewki, studni i beczki), a także częściej podejmował działania redukujące ryzyko niedoborów. Agent ze zmodyfikowanym promptem również realizował mechanizm motywacyjny, koncentrując się na trzech głównych działaniach (podlewanie roślin, ładowanie baterii, brak działania), lecz z ograniczoną adaptacyjnością i mniejszym wykorzystaniem synergii między zasobami.

Interpretacja wyników w kontekście teorii agentów motywowanych pozwoliła wykazać, że zarówno prosty, jak i zmodyfikowany wariant agenta LLM odzwierciedlał fundamentalne zasady zachowania motywacyjnego – działania podejmowane były przede wszystkim w odpowiedzi na sygnały krytyczne, a w sytuacjach neutralnych agent pozostawał bezczynny. Równocześnie analiza wykazała różnice w elastyczności i efektywności gospodarowania zasobami w zależności od jakości promptu, co stanowi istotny wniosek dla przyszłych badań nad integracją modeli językowych z mechanizmami motywacyjnymi.

Wkład własny autora koncentruje się zatem przede wszystkim na innowacyjnym podejściu badawczym, które poszerza wiedzę o zastosowaniu dużych modeli językowych w autonomicznych systemach decyzyjnych, przy jednoczesnym wykorzystaniu stworzonego kodu jako narzędzia analitycznego. Wyniki pracy stanowią podstawę do dalszych badań nad optymalizacją promptów i zwiększeniem adaptacyjności agentów opartych na LLM w środowiskach dynamicznych.

# Bibliografia

[1] Starzyk, J. A. (2008). Robotics, automation and control. [miejsce wydania: Ohio University U.S.A.].

[2] Starzyk, J. A., & Príncipe, J. C. (2017). Motivated learning for adaptive intelligent systems. IEEE Transactions on Neural Networks and Learning Systems, 28(3), 498–507.

[3] Pfeifer, R., & Bongard, J. (2007). How the body shapes the way we think: A new view of intelligence. MIT Press.

[4] Starzyk, J. A. (2014). Motivation in artificial systems: From survival instincts to abstract motivations. In Frontiers in Robotics and AI.

[5] Marsland, S. (2014). Machine learning: An algorithmic perspective. CRC Press.

[6] Sun, R. (2006). Cognition and multi-agent interaction: From cognitive modeling to social simulation. Cambridge University Press.

[7] Bubeck, S., et al. (2023). Sparks of Artificial General Intelligence: Early experiments with GPT-4. arXiv:2303.12712.

[8] Starzyk, J. A. (2009). Modeling for water resource management. [miejsce wydania: Ohio University, Athens 45701, USA].

[9] Starzyk, J. A. (2011). Motivated learning for computational intelligence. [miejsce wydania: Ohio University U.S.A.].

[10] Graham, J. (2015). Comparative study of motivated learning vs reinforcement learning. [miejsce wydania: Ohio Univ., Athens, OH, USA].

[11] Starzyk, J. A. (2012). Motivated learning for the development of autonomous agents. [miejsce wydania: Ohio University U.S.A.].

[12] Teng, Y. (2014). Integrating motivated learning and k-WTA to coordinate multi-agent reinforcement learning. [miejsce wydania: Ohio University U.S.A.].

[13] Starzyk, J. A. (2023). Motivated agent with semantic memory. [miejsce wydania: University of Information Technology and Management in Rzeszow, Rzeszow, Poland; Ohio University, Athens, OH 45701 USA].

[14] Starzyk, J. A. (2017). Needs, pains, and motivations in autonomous agents. IEEE Transactions on Neural Networks and Learning Systems, 28(11), 2523–2535.

[15] Xu, X. (2012). A biopsychically inspired cognitive system for intelligent agents in aerospace applications. In Proceedings of the Infotech@Aerospace 2012 Conference. American Institute of Aeronautics and Astronautics.

[16] Starzyk, J. A. (2017). MLECOG: Motivated Learning Embodied Cognitive Architecture. IEEE Systems Journal, 11(3), 1272–1283.

[17] Starzyk, J. A. (2010). Motivated learning for autonomous development. [miejsce wydania: School of Electrical Engineering and Computer Science at Ohio University, Athens, OH, USA].

[18] Starzyk, J. A. (2013). Simulation of motivated learning agent. [miejsce wydania: Ohio University, Athens 45701, USA].

[19] Graham, J. (2012). Transitioning from motivated to cognitive agent. [miejsce wydania: Ohio University, Athens 45701, USA].

[20] Starzyk, J. A. (2010). Water resource planning and management using motivated machine learning. In Hydrocomplexity: New Tools for Solving Wicked Water Problems (IAHS Publ. 338, pp. 21–29).

[21] Graham, J. T., & Starzyk, J. A. (2015). Opportunistic behavior in motivated learning agents. IEEE Transactions on Neural Networks and Learning Systems, 26(10), 2438–2451.

[22] Starzyk, J. A. (2016). Trust in motivated learning agents. [miejsce wydania: Ohio University, Athens 45701, USA].

[23] Teng, Y. (2014). Integrating motivated learning and k-WTA to coordinate multi-agent reinforcement learning. [miejsce wydania: Ohio University, Athens 45701, USA].

[24] Starzyk, J. A. (2023). Motivated agent with semantic memory – final report. [miejsce wydania: Rzeszów University of Technology, Poland; AGH University of Science and Technology in Kraków, Poland].

[25] Goodman, J. (2001). A bit of progress in language modeling. Computer Speech & Language.

[26] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.

[27] Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. IEEE Transactions on Neural Networks, 5(2), 157–166.

[28] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Polosukhin, I. (2017). Attention is all you need. arXiv preprint arXiv:1706.03762.

[29] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.

[30] OpenAI. (2018). Improving language understanding by generative pre-training (GPT-1). OpenAI.

[31] OpenAI. (2019). Language models are unsupervised multitask learners (GPT-2). OpenAI.

[32] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., … Amodei, D. (2020). Language models are few-shot learners (GPT-3). arXiv preprint arXiv:2005.14165.

[33] OpenAI. (2023). GPT-4 Technical Report. OpenAI.

[34] DeepSeek AI. (2024). DeepSeek-R1 technical overview. [Online]. Dostępne pod: https://www.deepseek.ai

[35] Google AI Blog; Meta AI Research Blog; Anthropic Publications (PaLM, LLaMA, Claude). (2023–2024). [Online].

[36] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., … Zhou, D. (2022). Emergent abilities of large language models. arXiv preprint arXiv:2206.07682.

[37] Bommasani, R., Hudson, D., Adeli, E., Altman, R., Arora, S., von Arx, S., … Liang, P. (2021). On the opportunities and risks of foundation models. Stanford Center for Research on Foundation Models.

[38] Baars, B. J., & Gage, N. M. (2007). Cognition, brain, and consciousness: Introduction to cognitive neuroscience. Elsevier.

[39] Derbyshire, S., et al. (1997); Melzack, R. (1990). [Źródła dotyczące teorii bólu].

[40] Russell, S., & Norvig, P. (2020). Artificial intelligence: A modern approach (4th ed.). Pearson.

[41] Kaelbling, L. P., Littman, M. L., & Moore, A. W. (1996). Reinforcement learning: A survey. Journal of Artificial Intelligence Research, 4, 237–285.

[42] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Hassabis, D. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529–533.

[43] Schmidhuber, J. (1991). A possibility for implementing curiosity and boredom in model-building neural controllers. In J.-A. Meyer & S. W. Wilson (Eds.), Proceedings of the International Conference on Simulation of Adaptive Behavior: From Animals to Animats (pp. 222–227). MIT Press.

[44] IBM. (2023). Reinforcement learning. Retrieved from https://www.ibm.com/think/topics/reinforcement-learning

[45] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., … Amodei, D. (2020). Language models are few-shot learners. Advances in Neural Information Processing Systems, 33, 1877–1901.

[46] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., … Polosukhin, I. (2017). Attention is all you need. In Advances in Neural Information Processing Systems (Vol. 30). Curran Associates, Inc.

[47] Christiano, P. F., Leike, J., Brown, T., Martic, M., Legg, S., & Amodei, D. (2017). Deep reinforcement learning from human preferences. Advances in Neural Information Processing Systems, 30.

[48] OpenAI. (2023). GPT-4 technical report. arXiv:2303.08774.

[49] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems, 35, 24824–24837.

[50] Bubeck, S., Chandrasekaran, V., Eldan, R., Gehrke, J., Horvitz, E., Kamar, E., … Zhang, Y. (2023). Sparks of artificial general intelligence: Early experiments with GPT-4. arXiv:2303.12712.

[51] Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., … Liang, P. (2021). On the opportunities and risks of foundation models. Stanford Center for Research on Foundation Models. arXiv:2108.07258.

[52] Ghosh, A., Kuo, Y.-H., Park, J. S., Baldi, P., & Honavar, V. (2022). Language as a cognitive tool: Connecting LLMs and reinforcement learning agents. arXiv:2211.11086.

[53] Graham, J. T., & Starzyk, J. A. (2012). Opportunistic motivated learning agents. In Proceedings of the International Joint Conference on Neural Networks (IJCNN) (pp. 1–8). IEEE.

[54] Graham, J. T., & Starzyk, J. A. (2014). Advancing motivated learning with goal creation. In Proceedings of the International Joint Conference on Neural Networks (IJCNN) (pp. 4047–4054). IEEE.

[55] Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2nd ed.). MIT Press.

[56] Barto, A. G., & Mahadevan, S. (2003). Recent advances in hierarchical reinforcement learning. Discrete Event Dynamic Systems, 13(4), 341–379.

[57] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., … Hassabis, D. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529–533.

[58] Sutton, R. S., Precup, D., & Singh, S. (1999). Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. Artificial Intelligence, 112(1–2), 181–211.

[59] Vezhnevets, A. S., Osindero, S., Schaul, T., Heess, N., Jaderberg, M., Silver, D., & Kavukcuoglu, K. (2017). FeUdal networks for hierarchical reinforcement learning. In Proceedings of the 34th International Conference on Machine Learning (ICML) (pp. 3540–3549). PMLR.

[60] Bacon, P.-L., Harb, J., & Precup, D. (2017). The option-critic architecture. In Proceedings of the AAAI Conference on Artificial Intelligence, 31(1). AAAI Press.

[61] Pfeifer, R., & Bongard, J. (2006). How the body shapes the way we think: A new view of intelligence. MIT Press.

[62] Backlund, A., & Petersson, L. (2025). Vending-Bench: A benchmark for long-term coherence of autonomous agents. Andon Labs.

[63] OpenAI. (2023). API reference: Parameter details (temperature). Retrieved from https://platform.openai.com/docs/api-reference/parameter-details

[64] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., … Amodei, D. (2020). Language models are few-shot learners. Advances in Neural Information Processing Systems, 33, 1877–1901.

[65] Karpathy, A. (2023). Intro to large language models [Video]. YouTube. https://www.youtube.com/watch?v=zjkBMFhNj_g

[66] Karpathy, A. (2023). Intro to large language models. YouTube.

[67] Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2nd ed.). MIT Press.

[68] Jurafsky, D., & Martin, J. H. (2021). Speech and language processing. Pearson.

[69] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.

[70] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., … Polosukhin, I. (2017). Attention is all you need. arXiv:1706.03762.

[71] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., … Amodei, D. (2020). Language models are few-shot learners. arXiv:2005.14165. Hotz, G. (2023). Public estimates on GPT-4 parameters. Various AI forums.

[72] Lake, B. M., & Murphy, G. L. (2023). Emergent behaviors in large language models. Journal of Artificial Intelligence Research.

[73] Dufter, P., Schmitt, M., & Schütze, H. (2021). Position information in transformers: An overview. arXiv:2102.11090.

[74] Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction. MIT Press.

[75] Ritter, F., Nerb, J., Lehtinen, E., & O'Shea, T. (2007). Instructed learning: Embodied agents and human learning. Springer.

[76] Zhou, H., Wang, Y., Chen, H., Zhang, Y., & Li, X. (2023). Language agent tree search unifies reasoning, acting, and planning in language models. arXiv:2305.17027.

[77] Wikipedia. (2023). Programowanie dynamiczne. Retrieved from https://pl.wikipedia.org/wiki/Programowanie_dynamiczne

[78] Ollama. (2023). Ollama Python client. GitHub repository. https://github.com/ollama/ollama-python

POLITECHNIKA RZESZOWSKA Rzeszów, 2025
im. Ignacego Łukasiewicza
WYDZIAŁ MATEMATYKI I FIZYKI STOSOWANEJ

STRESZCZENIE PRACY MAGISTERSKIEJ

Tytuł: Budowa agenta motywowanego na bazie Dużego Modelu Językowego (LLM)
Autor: inż. Klaudia Zawiślak
Promotor: dr inż. Marcin Kowalik, prof. PRz
Słowa klucze: Duże Modele Językowe (LLM), agent motywowany, uczenie ze wzmocnieniem, uczenie motywowane, sztuczna inteligencja.

Celem pracy było zbadanie potencjału dużych modeli językowych (LLM) jako komponentu decyzyjnego w autonomicznym agencie motywowanym. W Pythonie stworzono symulowane środowisko ogrodowe, w którym agent miał do dyspozycji 8 akcji, m.in. podlewanie roślin, ładowanie baterii i uzupełnianie zapasów wody. Przeprowadzono eksperyment porównawczy dwóch wariantów: agenta z podstawowym promptem oraz agenta ze zmodyfikowanym promptem, zawierającym rozbudowany opis stanu i sugestie działań; każda symulacja obejmowała 1000 iteracji. Wyniki wykazały, że obaj agenci skutecznie reagowali na bóle prymitywne zgodnie z mechanizmem „progu bólu”, jednak agent z podstawowym promptem był bardziej elastyczny i lepiej wykorzystywał synergię zasobów. Praca sugeruje, że LLM mogą pełnić rolę mechanizmu decyzyjnego w systemach motywowanych.

RZESZOW UNIVERSITY OF TECHNOLOGY Rzeszów 2025
FACULTY OF MATHEMATICS AND APPLIED PHYSICS

MASTER'S THESIS ABSTRACT

Title: Design of a Motivated Agent Based on a Large Language Model (LLM)
Author: inż. Klaudia Zawiślak
Supervisor: dr inż. Marcin Kowalik, prof. PRz
Key words: Large Language Models (LLMs), motivated agent, reinforcement learning, motivated learning, artificial intelligence.

The aim of this thesis was to investigate the potential of large language models (LLMs) as a decision-making component in an autonomous motivated agent. A simulated garden environment was implemented in Python, where the agent could choose from eight actions, including watering plants, charging the battery, and refilling water supplies. A comparative experiment was conducted between two variants: an agent with a basic prompt and an agent with a modified prompt, which included a detailed state description and suggested actions; each simulation consisted of 1000 iterations. The results showed that both agents effectively responded to primitive pains according to the "pain threshold" mechanism, but the agent with the basic prompt was more flexible and better utilized resource synergy. The study suggests that LLMs can serve as a decision-making mechanism in motivated systems.